

Index

Symbols

- `!=` operator
 - defined, [114](#)
 - example use, [114](#), [116](#)
- `"` (double-quote marks), [38](#)
- `"""` or `'''` (triple quotes), [38](#)
- `#` character, [39](#)
- `%` (remainder) operator
 - defined, [54](#), [60](#)
 - function of, [259](#)
- `%=` operator, [181](#)
- `&` operator, for intersection of sets, [468](#)
- `'` (single-quote marks), [37–38](#)
- `*` operator
 - multiplication, [54](#)
 - repetition, [345](#)
- `**` (exponent) operator, [54](#), [58–60](#)
- `*=` operator, [181](#)
- `+` operator
 - defined, [54](#)
 - displaying multiple items with, [68](#)
 - in list concatenation, [349](#)
- `+=` operator

- defined, [181](#)
 - in list concatenation, [350](#)
 - in string concatenation, [413](#)
- `-` (subtraction) operator, [54](#)
- `__str__` method, [507–510](#)
- `-=` operator, [181](#)
- `/` (division) operator, [54](#), [56](#)
- `//` (integer division) operator, [54](#), [56](#)
- `/=` operator, [181](#)
- `=` operator
 - defined, [112](#)
 - in subset determination, [470](#)
 - relationship testing, [113](#)
- `=` (assignment) operator, [113](#)
- `==` operator
 - assignment operator *versus*, [114](#)
 - defined, [112](#)
 - in string comparisons, [122](#)
 - use example, [116](#)
- `>` operator
 - defined, [112](#)
 - in string comparisons, [124](#)
 - use example, [112](#), [113–114](#)
- `>=` operator
 - defined, [112](#)
 - in superset determination, [470](#)
 - relationship testing, [113](#)

- `>>>` prompt, [21](#), [22](#)
- `^` operator, [469](#)
- `__init__` method, [499](#), [533](#), [539](#), [541](#), [561](#)
- `{}` (curly braces), [440](#)

A

- Accessor methods, [515](#)
- `acos(x)` function, [263](#)
- Actions
 - conditionally executed, [110](#)
 - in decision structures, [111](#)
- Ada programming language, [17](#)
- Addition `(+)` operator
 - defined, [54](#)
 - displaying multiple items with, [68](#)
 - precedence, [57](#)
- `add` method, [464](#)
- Algebraic expressions, [61–62](#)
- Algorithms
 - defined, [33](#)
 - example, [33](#)
 - recursive, [580](#), [581](#), [584–591](#)
- `and` operator
 - defined, [134](#)
 - false, [136](#)
 - short-circuit evaluation, [135](#)
 - truth table, [134](#)
- Appending data, [300](#)
- `append` method

- defined, [355–356](#)
 - use example, [356–357](#)
- Append mode, [300](#)
- Application software, [7](#)
- Arguments
 - defined, [225](#)
 - keyword, [233–235](#)
 - passing objects as, [518–519](#)
 - passing to functions, [225–235](#)
 - positional, [235](#)
 - as step values, [171](#)
- Arrays, [345](#)
- ASCII
 - character set, [11](#), [671](#)
 - defined, [11](#)
 - in string comparisons, [122–124](#)
- `asin` () function, [263](#)
- Assemblers, [15–16](#)
- Assembly language, [15–16](#)
- Assignment operators
 - augmented, [181–182](#)
 - `==` operator *versus*, [113](#)
- Assignment statements
 - creating variables with, [40–43](#)
 - example, [40](#)
 - format, [41](#)
 - multiple assignment, [451](#)
- `atan()` function, [263](#)

- Attributes. see **Data attributes**
- Augmented assignment operators, **181–182**
- `Automobile` class, **553–558**
- Averages
 - calculating, **58–59**
 - list value, **366–367**

B

- `BankAccount` class example, [505–510](#)
- bar chart
 - colors of, [394](#)
 - customizing width, [393–394](#)
 - plotting, [392–396](#)
- Base case, [581](#), [584](#), [586](#), [590](#)
- Base classes, [552](#)
- BASIC, [17](#)
- Binary files, [289](#)
- Binary numbering system, [9–10](#)
- Bits, [11](#)
 - defined, [8](#)
 - patterns, [9](#), [10](#)
- Black boxes, library functions as, [240](#)
- Blank lines, in blocks, [217](#)
- Blocks
 - blank lines, [217](#)
 - defined, [213](#)
 - indentation, [216–217](#)
 - nested, [128–129](#)
 - statement format, [213](#)
- `bool` data type, [139](#)
- Boolean expressions

- defined, [111–112](#)
- `if` statement testing, [111](#)
- with input validation loops, [185](#)
- with logical operators, [133](#)
- with relational operators, [111–112](#)
- Boolean functions
 - defined, [259](#)
 - in validation code, [260](#)
- Boolean values, returning, [259–260](#)
- Boolean variables, [139–140](#)
- Buffers, [293](#)
- Buttons
 - check, [626–628](#)
 - Quit, [610–611](#), [612](#)
 - radio, [622–625](#)
- `Button` widget
 - defined, [600](#)
 - use example, [608–611](#)
- Bytes
 - in data storage, [8](#)
 - defined, [7](#)
 - for large numbers, [10](#)

C

- Calculations
 - average, [58–59](#)
 - math expressions, [53](#)
 - math operators, [53](#), [54](#)
 - percentage, [55–56](#)
 - performing, [53–65](#)
- Callback functions
 - defined, [608](#)
 - as event handlers, [608](#)
 - `Radiobuttons` with, [625](#)
- Calling functions. see also [Functions](#)
 - defined, [36](#), [212](#)
 - examples of, [213](#)
 - general format, [36](#)
 - illustrated, [214](#)
- CamelCase, [44](#)
- `Canvas` widget, [600](#), [629–650](#)
 - `create_arc` method, [638–643](#)
 - `create_line` method, [631–633](#)
 - `create_oval` method, [636–638](#)
 - `create_polygon` method, [643–645](#)
 - `create_rectangle` method, [633–635](#)
 - `create_text` method, [645–650](#)

- screen coordinate system, [629–631](#)
- `Car` class, [540–541](#), [555–557](#)
- Case-sensitive comparisons, [423](#)
- `Cat` class, [568](#)
- `CD` class, [563](#)
- CD drives, [6](#)
- CDs (compact discs), [6](#)
- `ceil()` function, [263](#)
- Central processing unit (CPU)
 - as computer's brain, [13](#)
 - defined, [3](#)
 - fetch-decode-execute cycle, [14–15](#)
 - instruction set, [13](#)
 - microprocessor, [3–4](#)
 - operations, [13](#)
- `change_me` function, [231–233](#)
- Characters
 - encoding schemes, [11–12](#)
 - escape, [67–68](#)
 - extraction from string, [416–418](#)
 - individual, [408–412](#)
 - line continuation, [64](#)
 - newline, [66](#), [374](#)
 - in password, validating, [424–427](#)
 - selecting range of, [415–416](#)
 - space, [51](#)
 - storage, [11](#)
 - string, accessing, [408–412](#)

- whitespace, [420](#), [422](#)
- Character sets, ASCII, [671](#)
- Charts, IPO, [254–255](#)
- Checkbuttons, [600](#), [626–628](#)
- `Checkbutton` widget
 - creating, [626](#)
 - defined, [600](#)
 - use example, [626–628](#)
- Class definitions
 - `Coin` example, [495–501](#)
 - defined, [494](#)
 - organization, [503](#)
- Classes. see also [Instances](#)
 - `Automobile`, [553–558](#)
 - `BankAccount`, [505–510](#)
 - base, [552](#)
 - `Car`, [540–541](#), [555–557](#)
 - `Cat`, [568](#)
 - `CD`, [563](#)
 - `CellPhone`, [513–515](#)
 - `Checkbutton`, [626–628](#)
 - `Coin`, [495–501](#)
 - concept of, [493](#)
 - `Contact`, [522–532](#)
 - cookie cutter metaphor, [494](#)
 - `Customer`, [539–540](#)
 - defined, [493](#)

- derived, [552](#)
- designing, [532–543](#)
- `Dog`, [567–568](#)
- finding, in problems, [533–538](#)
- inheritance, [551–572](#)
- `IntVar`, [623](#)
- `Mammal`, [566–568](#)
- in modules, storing, [503–505](#)
- polymorphism, [566–572](#)
- `Radiobutton`, [622–625](#)
- responsibilities, identifying, [538–543](#)
- `SavingsAccount`, [562](#), [563](#)
- `ServiceQuote`, [541–543](#)
- `SUV`, [553](#), [558–560](#)
- `Truck`, [557–560](#)
- UML layout, [533](#)
- Code
 - debugging, [132](#)
 - defined, [19](#)
 - modularized program, [210](#)
 - responding to exceptions, [330–332](#)
 - validation, [260](#)
- Colors, predefined names, [673–677](#)
- Command line interfaces, [597–599](#)
- Comments
 - defined, [39](#)
 - end-line, [39–40](#)

- importance of, [40](#)
- Compilers, [18–19](#)
- Computers
 - components, [2–7](#)
 - CPU, [3–4](#)
 - data storage, [7–12](#)
 - ENIAC, [3](#), [4](#)
 - input devices, [6](#)
 - main memory, [5](#)
 - output devices, [6](#)
 - secondary storage devices, [5–6](#)
 - user interface, [597](#)
- Concatenation
 - defined, [412](#)
 - list, [349–350](#)
 - newline to string, [297–303](#)
 - `+=` operator in, [350](#), [413](#)
 - string, [68](#), [412–413](#)
- Conditional execution
 - `if-else` statement, [119](#)
- Condition-controlled loops. see also [Loops](#)
 - defined, [160](#), [196](#)
 - `while` loop, [160–168](#)
- `Contact` class example, [522–532](#)
- Continuation character, [64](#)
- Control structures, [109](#), [125](#)
- Cookies, [288](#), [494](#)
- Copying lists, [362–364](#)

- `cos()` function, [263](#)
- Count-controlled loops, [160](#), [168–178](#). see also [Loops](#)
- `create_arc` method, [638–643](#)
 - optional arguments to, [640](#)
- `create_line` method, [631–633](#)
 - optional arguments to, [633](#)
- `create_oval` method, [636–638](#)
 - optional arguments to, [637](#)
- `create_polygon` method, [643–645](#)
 - optional arguments to, [645](#)
- `create_rectangle` method, [633–635](#)
 - optional arguments to, [635](#)
- `create_text` method, [645–650](#)
 - optional arguments to, [647](#)
- C# programming language, [17](#)
- `Customer` class, [539–540](#)

D

- Data
 - appending to files, [300](#)
 - binary file, [289](#)
 - digital, [12](#)
 - hiding, [490](#)
 - output, [65–73](#)
 - processing, [311–324](#)
 - reading from files, [288–289](#), [294–297](#)
 - text file, [289](#)
 - writing to files, [288](#), [292–294](#)
- Data attributes, [490](#)
 - hidden, [491](#)
 - hiding, [500–503](#)
 - initializing, [496](#)
 - instance, [510–532](#)
- Data storage, [7–11](#)
 - characters, [11](#)
 - music, [12](#)
- Data types
 - `bool`, [139](#)
 - conversion and mixed-type expressions, [63–64](#)
 - and literals, [46–47](#)
 - mixing, [444–445](#)

- `str`, [47–48](#)
- Debugging, [32](#), [132](#)
 - global variables and, [236–237](#)
- Decision structures, [110](#)
 - actions performed in, [110–111](#)
 - combining sequence structures with, [125–126](#)
 - dual alternative, [118](#)
 - in flowcharts, [110](#)
 - `if-elif-else` statement, [131–132](#)
 - `if-else` statement, [118–121](#)
 - `if` statement, [109–117](#)
 - inside, [126](#)
 - nested, [125–133](#)
 - single alternative, [110](#)
- `degrees()` function, [263](#)
- `del` statement, [361](#)
- Depth of recursion, [580](#)
- Derived classes, [552](#)
- Dialog boxes
 - defined, [597](#)
 - illustrated, [598–599](#)
 - info, [608–611](#)
- Dictionaries
 - creating, [440](#)
 - defined, [439](#)
 - elements, adding, [442](#)
 - elements, deleting, [443](#), [447](#)
 - elements, display order, [440](#)

- elements, number of, [443–444](#)
- empty, creating, [445–446](#)
- example, [452–455](#), [455–461](#)
- iteration, [446–447](#)
- keys, [439](#), [440](#)
- key-value pairs, [439](#), [442](#)
- methods, [447–452](#)
- mixing data types in, [444–445](#)
- parts of, [439](#)
- retrieving value from, [440–441](#)
- storing objects in, [522–532](#)
- values, [441–442](#)
- Dictionary views
 - defined, [448](#)
 - elements, [448](#)
 - keys returned as, [449](#)
 - values returned as, [451–452](#)
- `dict()` method, [446](#)
- `difference` method, [468–469](#)
- Digital data, [12](#)
- Digital devices, [12](#)
- Direct access files, [290](#), [321](#), [324](#)
- Direct recursion, [584](#)
- `discard` method, [465](#)
- Disk drives
 - defined, [5](#)
 - program storage on, [14](#)
- Divide and conquer approach, [210](#)

- Division
 - floating-point, [56](#)
 - integer, [54](#), [56](#)
 - / operator, [54](#), [56](#), [57](#)
- `Dog` class, [567–568](#)
- Dot notation, [241](#), [263](#)
- Dual alternative decision structure, [117–118](#)
- `dump` method, [475](#)
- DVD drives, [6](#)
- DVDs (digital versatile discs), [6](#)

E

- `else` clause, [121](#), [128](#)
 - execution, [118](#)
 - with `try/except` statement, [335–336](#)
- `else` suite, [335](#)
- Encapsulation, [490](#)
- `end-of-file`, [478–479](#), [521](#)
- `endswith` method, [423](#), [424](#)
- ENIAC computer, [3](#), [4](#)
- `Entry` widget
 - defined, [600](#), [611](#)
 - getting input with, [611–614](#)
 - use example, [612–614](#)
- Error handlers, [187](#)
- Error messages, [19](#), [22](#)
 - default, displaying, [333–334](#)
 - loop displaying, [186–187](#)
 - traceback, [327](#), [329](#)
- Errors. see [Exceptions](#)
- Error traps, [187](#)
- Escape characters, [67–68](#)
- Event handlers, callback functions as, [608](#)
- Exception handling
 - defined, [330](#)

- multiple exceptions, [330–333](#)
- with `try/except` statement, [335–336](#)
- Exception objects, [337](#)
- Exceptions
 - catching all, with one `except` clause, [332–333](#)
 - code response to, [326](#)
 - default error message display, [333–334](#)
 - defined, [53](#), [324](#)
 - `IndexError`, [346–347](#), [411](#)
 - `IOError`, [329–330](#), [332](#), [336](#)
 - `KeyError`, [441](#), [443](#), [465](#)
 - multiple, [330–332](#)
 - unhandled, [336–337](#)
 - `ValueError`, [327–328](#), [332](#), [333](#), [334](#), [336](#), [360](#)
 - `ZeroDivisionError`, [336](#)
- Execution, [201](#)
 - in fetch-decode-execute cycle, [14](#)
 - pausing, [222](#)
 - Python interpreter, [664–665](#)
 - `try/except` statement, [327](#)
 - `while` loop, [187](#)
- Exercises
 - decision structures, [151–157](#)
 - dictionaries and sets, [485–488](#)
 - files and exceptions, [344–346](#)
 - functions, [280–286](#)
 - GUI programming, [654–657](#)

- inheritance, [574–575](#)
- input, processing, and output, [104–108](#)
- introduction, [28–29](#)
- lists and tuples, [402–406](#)
- programming, [546–550](#)
- recursion, [594–595](#)
- repetition structures, [243–245](#)
- strings, [434–438](#)
- `exp()` function, [263](#)
- Exponent (**) operator, [54](#), [58–60](#)

F

- `Factorial` function, [582–583](#)
- Factorials
 - calculation with recursion, [580–584](#)
 - definition rules, [584](#), [587](#)
- Fetch-decode-execute cycle, [14–15](#)
- Fibonacci series
 - defined, [585–586](#)
 - recursive function calculation, [586–587](#)
- Field widths
 - columns, [71](#)
 - minimum, specifying, [71–72](#)
- File modes, [295](#)
- Filename extensions, [290](#)
- File objects
 - `close` method, [293](#), [475](#)
 - defined, [290](#)
 - `read` method, [294–297](#)
 - variable name referencing, [290–292](#)
 - `writelines` method, [373–376](#)
 - `write` method, [292](#)
- `finally` clause, with `try/except` statement, [336](#)
- Finally suite, [336](#)
- `find` method, [424](#)

- Flags, **140**
- Flash drives, **6**
- Flash memory, **6**
- `float` data type, **47, 63–64**
- `float()` function, **51, 52, 53, 64**
- Floating-point division, **56, 57**
- Floating-point notation, **11**
- Floating-point numbers, **72**
- `floor()` function, **263**
- Floppy disk drives, **5**
- Flowchart, **34**
 - clock simulator, **191**
 - decision structure, **110, 111, 115**
 - dual alternative decision structure, **108**
 - input validation loop, **186**
 - logic for file end detection, **306**
 - nested decision structure, **127, 130**
 - pay calculating program, **35**
 - program development cycle, **31**
 - running total calculation, **179**
 - sequence structure nested inside decision structure, **126**
 - sequence structures with decision structure, **125–126**
 - `while` loop, **161**
- `Font` class, **648–650**
 - keyword arguments for, **649**
- `for` loops. see also **Loops**
 - as count-controlled loops, **168–178**
 - defined, **168–169**

- in turtle graphics, [197](#)
- iterating lists with, [346](#)
- iterating over dictionaries with, [446–447](#)
- iterating over list of strings with, [170](#)
- iterating over sets, [466](#)
- iterations, [176–178](#)
- iterations, user control, [176–178](#)
- `range` function with, [168–172](#)
- target variables, [172–176](#)
- `format` function, [69](#), [72](#)
- Format specifiers, [69–72](#)
- Formatting
 - with comma separators, [70–71](#)
 - floating-point number as percentage, [72](#)
 - integers, [72–73](#)
 - numbers, [68–69](#)
 - in scientific notation, [70](#)
- `for` statement, [168](#)
- FORTRAN, [17](#)
- `Frame` widget, [600](#), [607](#)
- Function call
 - defined, [213](#)
 - examples of, [214](#), [215](#)
 - in flowcharts, [218](#)
 - nested, [52](#)
 - process, [213–216](#)
 - recursive, [580](#)
- Function definitions

- block, [214](#)
- defined, [212](#)
- function header, [212](#)
- general format, [212–213](#)
- Function headers, [212](#), [216](#), [226](#), [230](#)
- Functions, [361–362](#)
 - `acos()`, [263](#)
 - `asin()`, [263](#)
 - `atan()`, [263](#)
 - Boolean. see [Boolean functions](#)
 - callback. see [Callback functions](#)
 - calling. see [Calling functions](#)
 - `ceil()`, [263](#)
 - `cos()`, [263](#)
 - `degrees()`, [263](#)
 - `exp()`, [263](#)
 - `factorial`, [582–583](#)
 - `float()`, [51](#), [52](#), [53](#), [64](#)
 - `floor()`, [263](#)
 - `format`, [69](#), [72](#)
 - `hypot()`, [263](#)
 - importing, [679–680](#)
 - `input`, [49–52](#)
 - `int()`, [51–53](#)
 - `is_even`, [259](#)
 - `isinstance`, [569–572](#)

- `len`, [347](#), [412](#), [443–444](#)
- `library`, [239–241](#)
- `list()`, [344–345](#), [382](#)
- `load`, [475](#), [476](#)
- `log()`, [263](#)
- `message`, [213–218](#), [577–579](#)
- `min`, [361–362](#)
- `open`, [291](#), [292](#), [296](#)
- `print.` see `print` function
- `radians()`, [263](#)
- `randint`, [241–244](#), [246](#)
- `range.` see `range` function
- `range_sum`, [584–585](#)
- `recursive.` see [Recursive functions](#)
- `set`, [463](#)
- `sin()`, [263](#)
- `showinfo`, [608](#)
- `sqrt()`, [263](#)
- `sum`, [251–252](#)
- `tan()`, [263](#)
- `type`, [47](#)
- `value-returning.` see [Value-returning functions](#)

G

- GCD (greatest common divisor), [587–588](#)
- Generalization, [551–552](#)
- `get` method, [448](#), [528](#), [611–612](#), [614](#), [617](#), [625](#)
- Getters, [515](#)
- Global constants, [379](#)
 - defined, [237](#)
 - using, [237–239](#)
- Global variables. see also [Variables](#)
 - accessing, [235](#)
 - creating, [236](#)
 - debugging and, [236–237](#)
 - defined, [235](#)
 - examples of, [235–236](#)
 - functions using, [237](#)
 - program understanding and, [237](#)
 - use restriction, [236–237](#)
- Greatest common divisor (GCD), [587–588](#)
- GUI programs
 - `Button` widgets, [608–611](#)
 - check buttons, [626–628](#)
 - creating, [599–602](#), [618](#)
 - `Entry` widget, [611–614](#)
 - as event-driven, [599](#)

- fields, **614–618**
- getting input with, **611–614**
- info dialog boxes, **608–611**
- `Label` widgets, **602–605**
- radio buttons, **622–625**
- sketch of window, **618–619**
- text display, **602–605**
- with `tkinter` module, **599–602**
- widget list, **619**
- widget organization, **605–608**

H

- Hardware, [2–7](#)
 - components, [2–3](#)
 - defined, [2](#)
- Hierarchy charts, [219–220](#), [228](#). see also [Program design](#)
- High-level programming languages, [16–18](#)
- `hypot()` function, [263](#)

- IDLE. see [Integrated development environment](#)
- `if-elif-else` statement, [131–132](#). see also [Decision structures](#)
- `if-else` statement. see also [Decision structures](#)
 - conditional execution in, [119](#)
 - defined, [117](#)
 - general format, [118](#)
 - indentation, [119](#)
 - nested, [132](#), [136](#)
 - use of, [119–120](#)
- `if` statement, [109](#). see also [Decision structures](#)
 - Boolean expression testing, [111–114](#)
 - execution, [109](#)
 - general format, [109](#)
 - nested blocks, [116–117](#)
 - use example, [114–116](#)
 - use of, [116–117](#)
- `import` statement, [240–241](#), [244](#), [679–681](#)
 - and alias, [681](#)
 - wildcard, [680](#)
- `import turtle` statement, [75](#)
- Indentation
 - IDLE, automatic, [667–668](#)
 - IDLE, default, [668](#)

- `if-else` statement, [119](#)
- nested decision structures, [128](#)
- in Python, [216–217](#)
- `IndexError` exception, [346–348](#), [411](#). see also [Exceptions](#)
- Indexes
 - characters, [410](#)
 - defined, [346](#)
 - invalid, [353](#), [356](#), [411](#), [416](#)
 - with lists, [347–348](#)
 - negative, [346](#), [353](#), [356](#), [411](#), [416](#)
 - position reference, [353](#), [416](#)
 - in slicing expression, [352](#)
 - string, [410–411](#)
 - tuples support, [381](#)
- `index` method, [357–359](#)
- Indirect recursion, [584](#)
- Infinite loops, [167](#)
- Info dialog boxes, [608–609](#), [611](#), [614–615](#), [625–626](#)
- Inheritance
 - defined, [551](#)
 - generalization and, [551](#)
 - and “is a” relationship, [552–560](#)
 - specialization and, [551](#)
 - in UML diagrams, [560–561](#)
 - using, [561–565](#)
- Initializer methods, [496](#)
- `in` operator
 - general format, [354](#)

- searching list items with, [354–355](#)
- testing dictionary values with, [441–442](#)
- testing set values with, [466–467](#)
- testing strings with, [419](#)
- Input
 - in computer program process, [35–36](#)
 - defined, [6](#)
 - with `Entry` widget, [611–614](#)
 - flowchart symbols, [34](#)
 - reading from keyboard, [49–53](#)
- Input devices, [6](#)
- Input files, [288–289](#)
- `input` function, [49–52](#)
- Input validation loops, [185–189](#). see also [Loops](#)
- `insert` method, [359](#)
- Instances, working with, [510–532](#)
- Instructions, [13–15](#)
- Instruction sets, [13](#), [15](#)
- Integer division, [54](#), [56–57](#)
- Integers
 - formatting, [72–73](#)
 - list of, [344](#)
 - `randint` function return, [246](#)
 - `randrange` function return, [248](#)
- Integrated development environment (IDLE), [24](#)
 - automatic indentation, [667–668](#)
 - color coding, [667](#)
 - defined, [23](#)

- indentation, [216–217](#)
- Python Shell window, [663–665](#), [667](#), [668–669](#)
- resources, [670](#)
- running programs in, [668–669](#)
- saving programs in, [668](#)
- shell window, [664](#)
- software bundling, [663](#)
- starting, [663–665](#)
- statement execution, [664](#)
- text editor, [23](#)
- `tkinter` module, [599](#)
- writing Python programs in, [665–666](#)
- Interactive mode, [21–23](#), [36](#), [47](#), [53](#), [66](#), [69](#), [112](#), [244–245](#), [349–350](#), [353](#)
- Interpreters
 - defined, [18](#)
 - high-level program execution, [19](#)
 - Python, [20](#)
- `intersection` method, [468](#)
- Intersection of sets, [468](#)
- `int()` function, [51–53](#)
- `IntVar` class, [623](#)
- `IOError` exceptions, [329–330](#), [332](#), [336](#)
- IPO charts, [54–255](#)
- `isalnum()` method, [420](#)
- `isalpha()` method, [420](#)
- `isdigit()` method, [420](#)

- `is_even` function, **259**
- `isinstance` function, **569–572**
- `islower()` method, **420**
- `isspace()` method, **420**
- `issubset` method, **470**
- `issuperset` method, **470**
- `isupper()` method, **420**
- item separators, **66–67**
- `items` method, **448–449**
- Iterable, **170–171**, **178**, **195–196**, **344–345**
- Iteration, loop, **167**, **170**
 - nested loops, **195–196**
 - and sentinel, **183**
 - user control, **176–178**

J

- Java, [17](#)
- JavaScript, [17](#)

K

- Keyboard, reading input from, [49–53](#)
- `KeyError` exceptions, [441](#), [443](#), [465](#), [466](#)
- Keys. see also [Dictionaries](#)
 - defined, [439](#)
 - dictionary views and, [448–449](#)
 - different types of, [445](#)
 - duplicate, [442](#)
 - string, [440](#)
 - types of, [440](#)
- `keys` method, [449](#), [462](#)
- Key-value pairs
 - adding, [442](#)
 - defined, [439](#)
 - deleting, [443](#)
 - as mappings, [440](#)
 - removing, [450](#), [451](#)
 - returning, [451](#)
- Keyword arguments, [233–235](#)
- Key words, [16–17](#), [32](#), [43](#), [212–213](#), [265](#)

L

- `Label` widget
 - creating, [607](#)
 - defined, [600](#)
 - as output fields, [614–618](#)
 - `pack` method, [603](#)
 - stacked, [604](#)
 - text display with, [602–605](#)
- Latin Subset of Unicode, [671](#)
- `len` function, [347](#), [412](#), [443–444](#)
- Library functions, [239–241](#)
- Line graph
 - adding title/axis labels/grid, [385–386](#)
 - data points, displaying markers at, [390–391](#)
 - plotting, [384–391](#)
 - X/Y axis, customizing, [386–390](#)
- `Listbox` widget, [600](#)
- `list()` function, [344–345](#), [382](#)
- Lists
 - accepting as an argument, [361](#), [362](#)
 - `append` method and, [355–357](#)
 - concatenating, [349–350](#)
 - contents, displaying, [379](#)
 - converting iterable objects to, [344–345](#)

- converting to tuples, [382](#)
- copying, [362–364](#)
- defined, [212](#), [343](#), [344](#)
- of different types, [344](#)
- displaying, [344](#)
- as dynamic structures, [343](#)
- elements, [344](#)
- finding items in, [354–355](#)
- indexing, [346–347](#)
- `index` method and, [357–359](#)
- `insert` method and, [359](#)
- of integers, [244](#)
- items, adding, [355–357](#)
- items, in math expressions, [364–365](#)
- items, inserting, [359](#)
- items, removing, [360–361](#)
- items, reversing order of, [361](#)
- items, sorting, [359–360](#)
- iterating with `for` loop, [346](#)
- as mutable, [347–349](#)
- passing as argument to functions, [367–368](#)
- processing, [364–376](#)
- recursion, [584–585](#)
- `remove` method and, [360–361](#)
- returning from functions, [368–370](#)
- `reverse` method and, [361](#)
- slicing, [351–354](#)
- `sort` method and, [359–360](#)

- storing objects in, [516–518](#)
- of strings, [344](#)
- tuples *versus*, [380](#)
- two-dimensional, [376–380](#)
- values, totaling, [366](#)
- working with, [373–376](#)
- writing to files, [375](#)
- Literals
 - numeric, [47](#)
 - string literals, [37–38](#), [45](#), [67–68](#), [173](#), [292](#), [297](#), [408](#), [419](#)
- `load` function, [475](#), [476](#)
- Local variables, [223–225](#), [232](#), [261](#), [580](#). see also [Functions](#); [Variables](#)
- `log()` function, [263](#)
- Logical operators
 - and, [134–136](#)
 - defined, [133](#)
 - expressions with, [133](#)
 - `not`, [133](#), [135](#)
 - numeric ranges with, [138](#)
 - `or`, [133–135](#)
 - types of, [133](#)
- Logic errors, [32](#)
- Loops
 - `for`, [168–178](#), [307–311](#), [346](#), [408](#), [446–447](#), [449](#), [452](#), [466](#)
 - condition-controlled, [160–168](#)
 - count-controlled, [168–178](#)
 - end of files and, [478–479](#)

- in turtle graphics, **197–200**
- infinite, **167**
- input validation, **229–234**
- iteration, **167, 176, 183**
- nested, **190–196, 379**
- pretest, **164–165**
- priming read, **186**
- recursion *versus*, **591**
- sentinels and, **182–184**
- validation, **260**
- `while`, **160–168**
- `lower()` method, **422–423**
- Low-level languages, **16**
- `rstrip()` method, **422**

M

- Machine language, [12–16](#), [18–19](#), [32](#)
- Main memory
 - defined, [5](#)
 - programs copied into, [14–15](#)
- `Mammal` class, [566–568](#)
- Math expressions
 - algebraic, [61–62](#)
 - defined, [53](#)
 - list elements in, [364–365](#)
 - mixed-type, [63–64](#)
 - operands, [54](#)
 - parentheses, grouping with, [58–59](#)
 - `randint` function in, [241–242](#)
 - simplifying with value-returning functions, [250–254](#)
- `Math` module, [261–264](#)
- Math operators
 - defined, [53](#)
 - list of, [54](#)
- `matplotlib` package, [383–398](#)
 - bar chart, plotting, [392–396](#)
 - line graph, plotting, [384–391](#)
 - pie chart, plotting, [396–398](#)
 - `pyplot` module, [383](#)

- `max` function, [361–362](#)
- Memory
 - buffer, [293](#)
 - flash, [6](#)
 - main memory, [5](#)
 - random access (RAM), [5](#)
- Memory sticks, [6](#)
- `Menubutton` widget, [600](#)
- Menu-driven programs, [268](#)
- Method overriding, [566](#)
- Microprocessors, [3–4](#)
- `min` function, [361–362](#)
- Mixed-type expressions, [63–64](#)
- Mnemonics, [15](#)
- Modification methods, [422–423](#)
- Modifying records, [319–320](#)
- Modularization, [264](#)
- Modularized programs, [210–211](#)
- Modules
 - benefits of, [264](#)
 - defined, [240](#), [264](#)
 - function, [255–258](#)
 - importing, [265](#)
 - `math`, [261–263](#)
 - `pickle`, [475](#)
 - `random`, [241](#), [244](#), [247](#)
 - `rectangle`, [265–266](#)

- storing classes in, [503–505](#)
- storing functions in, [264–268](#)
- `tkinter`, [599–602](#)
- using, [265](#)
- Modulus operator. see [Remainder \(%\) operator](#)
- Multiple assignment, [451](#)
- Multiple items
 - displaying with + operator, [68](#)
 - displaying with print function, [45](#)
 - separating, [66–67](#)
- Multiplication (*) operator, [54](#), [57](#), [345](#)
- Mutable, lists as, [347–349](#)
- Mutator methods, [515](#)

N

- Named constant, [73–74](#)
- Negative indexes, [346](#), [353](#), [411](#), [416](#)
- Nested blocks, [128–129](#)
- Nested decision structures, [127–132](#)
- Nested function calls, [52](#)
- Nested lists. see [Two-dimensional lists](#)
- Nested loops, [190–196](#), [379](#). see also [Loops](#)
- Newline `(\n)` character
 - concatenating to a string, [297–298](#)
 - stripping from string, [298–300](#), [374](#)
 - suppressing, [65–66](#)
- `not in` operator
 - in list item determination, [355](#)
 - testing dictionary values with, [441–442](#)
 - testing set values with, [466–467](#)
 - testing strings with, [419](#)
- `not` operator, [133](#), [135](#)
- Nouns, list refining, [535–538](#)
- Numbers
 - advanced storage, [12](#)
 - binary, [9–10](#)
 - factorial of, [581–583](#)
 - Fibonacci, [585–587](#)

- floating-point, [11](#), [72](#)
- formatting, [68–69](#)
- pseudorandom, [248–249](#)
- random, [240–249](#)
- running totals, [179–182](#)
- storage, [9–11](#)
- Numeric literals, [47](#)
- Numeric ranges, with logical operators, [138](#)

O

- Object-oriented programming (OOP)
 - class design, [532–543](#)
 - classes, [493–510](#)
 - data hiding, [490](#)
 - defined, [490](#)
 - encapsulation, [490](#)
 - instances, [493](#), [510–532](#)
- Objects, [607](#). see also [Classes](#)
 - characteristics description, [493](#)
 - defined, [490](#)
 - dictionary, [439–462](#)
 - everyday example of, [491–492](#)
 - exception, [337](#)
 - file, [294–295](#)
 - list, [343](#), [344](#)
 - “is a” relationship, [552](#)
 - passing as arguments, [518–519](#)
 - pickling, [520–522](#)
 - reusability, [491](#)
 - sequence, [343](#)
 - serializing, [474–480](#)
 - set, [462–474](#)
 - storing in dictionaries, [522–532](#)
 - storing in lists, [516–518](#)

- `StringVar`, **614–615**
 - unpickling, **521–522**
- `open` function, **291, 292, 296**
- Operands, **54**
- Operating systems, **6**
- Operators
 - `and`, **134–136**
 - `in.` see `In` operator
 - addition (+), **54, 57, 68**
 - assignment, **113, 181–182**
 - augmented assignment, **181–182**
 - exponent (**), **54, 58–60**
 - logical. see **Logical operators**
 - math, **53–54**
 - multiplication (*), **54, 57, 345**
 - `not in.` see `Not in` operator
 - `or`, **133–135, 137, 138**
 - precedence, **57**
 - relational, **111–112, 123–124**
 - remainder (%), **54, 57, 60–61**
 - repetition (*), **345, 428–429**
 - subtraction (–), **54, 57**
- Optical devices, **6**
- `or` operator, **133–135, 137, 138**
- Output
 - in computer program process, **35–36**
 - data, **65–73**

- defined, [6](#)
 - displaying, [36–39](#)
- Output devices, [6](#)
- Output files, [288](#), [289](#), [293](#), [300](#), [304](#)
- Overhead, [580](#)
- Overriding, method, [566](#)

P

- `pack` method, [603–605](#), [607](#), [612](#)
- Parameter lists, [230](#)
- Parentheses, grouping with, [58](#)
- Pascal, [17](#)
- Passing arguments. see also [Arguments](#)
 - to function, [226–229](#)
 - function parameter variable, [231–233](#)
 - `to index` method, [357–359](#)
 - list as function, [367–368](#)
 - multiple, [229–231](#)
 - and objects, [518–519](#)
 - by position, [230](#)
 - by value, [233](#)
- Passwords, validating characters in, [424–427](#)
- Pattern printing using nested loops, [193–196](#)
- Percentages, calculations, [55–56](#)
- `Pickle` module, [475–476](#), [520](#), [525](#)
- Pickling, [474–476](#), [478](#), [520–522](#)
- `pi` variable, [263](#)
- pie chart, plotting, [396–398](#)
- `pip` utility, [683](#)
- Pixels, [12](#)
- `plot()` function, [384–385](#)

- Polymorphism
 - behavior, ingredients of, [566](#)
 - defined, [566](#)
 - `isinstance` function, [569–572](#)
- `popitem` method, [451](#), [455](#)
- `pop` method, [450](#), [698](#)
- Positional arguments, [235](#)
- Precedence, operator, [57](#)
- Predefined names colors, [673–677](#)
- Pretest loops
 - defined, [164](#)
 - `while` loops as, [164–165](#)
- Priming reads, [186](#), [308](#)
- `print` function
 - ending newline suppression, [65–66](#)
 - multiple item display with, [45](#)
 - output display with, [36–39](#)
- Private methods, [492](#)
- Problem domain
 - defined, [534](#)
 - nouns/noun phrases in, [534–535](#)
 - writing description of, [534–535](#)
- Problem solving
 - base case, [581](#)
 - with loops, [580](#)
 - with recursion, [580–584](#)
 - recursive case, [581](#)
- Procedural programming

- defined, [489](#)
 - focus of, [489](#)
 - vs. object-oriented programming, [490](#)
- Procedures, [373](#), [489–490](#)
- Processing files, [293](#)
- Processing lists, [364–376](#)
 - an argument to function, [367–368](#)
 - average of values in, [366–367](#)
 - and files, [373–376](#)
 - from function, returning, [378–370](#)
 - total of values in, [376](#)
- Processing symbols, [34](#)
- Program design, [31–35](#)
 - in development cycle, [31–32](#)
 - flowcharts, [34–35](#), [217–218](#)
 - with functions, [217–222](#)
 - hierarchy charts, [219](#)
 - with `for` loops, [218–220](#)
 - pseudocode, [34](#)
 - steps, [31–32](#)
 - task breakdown, [32–33](#)
 - top-down, [218–219](#)
 - with `while` loops, [165–167](#)
- Program development cycle, [31–32](#)
- Programmers
 - comments, [39–40](#)
 - customer interview, [33](#)
 - defined, [1](#)

- task breakdown, 33
- task understanding, 33
- Programming
 - GUI, 597–657
 - in machine language, 15–16
 - object-oriented (OOP), 489–550
 - procedural, 489–490
 - with Python language, 1–2
- Programming languages, 17, 597–657. See also **Python**
 - Ada, 17
 - assembly, 15
 - BASIC, 17
 - C/C++, 17
 - COBOL, 17
 - FORTRAN, 17
 - high-level, 16–17
 - Java, 17
 - JavaScript, 17
 - low-level, 16
 - Pascal, 17
 - Ruby, 17
 - Visual Basic, 17
- Programs
 - assembly language, 15
 - compiling, 18–19
 - copied into main memory, 14–15
 - defined, 1
 - execution, pausing, 222

- flowcharting, with functions, **217–218**
- functioning of, **12–19**
- image editing, **2**
- menu driven, **268**
- modularized, **210–211**
- running, **3**, **668–669**
- saving, **668**
- storage, **14**
- testing, **20**
- transfer, control of, **216**
- utility, **6**
- word processing, **2**, **5**, **7**
- writing in IDLE editor, **23**, **665–666**
- Programs (examples in this book)
 - `account_demo.py`, **564–565**
 - `accounts.py`, **562**, **563**
 - `account_test.py`, **506–507**
 - `account_test2.py`, **509–510**
 - `acme_dryer.py`, **221–222**
 - `add_coffee_record.py`, **315–316**
 - `animals.py`, **566–567**, **568**
 - `auto_repair_payroll.py`, **120**
 - `average_list.py`, **367**
 - `bad_local.py`, **223**
 - `bankaccount.py`, **505**
 - `bankaccount2.py`, **508–509**
 - `barista_pay.py`, **364–365**

- `birds.py`, **224–225**
- `birthdays.py`, **456–461**
- `button_demo.py`, **608–609**
- `card_dealer.py`, **453–455**
- `car_demo.py`, **556–557**
- `car.py`, **541**
- `car_truck_suv_demo.py`, **559–560**
- `cell_phone_list.py`, **516–518**
- `cellphone.py`, **513–514**
- `cell_phone_test.py`, **513–515**
- `change_me.py`, **231–232**
- `checkboxbutton_demo.py`, **626–628**
- `circle.py`, **264–265**
- `coin_argument.py`, **519**
- `coin_demo1.py`, **497–498**
- `coin_demo2.py`, **500–501**
- `coin_demo3.py`, **501–503**
- `coin_demo4.py`, **504**
- `coin_demo5.py`, **511**
- `coin.py`, **503–504**
- `coin_toss.py`, **246–247**
- `columns.py`, **71–72**
- `commission.py`, **162**
- `commission2.py`, **210**
- `commission_rate.py`, **256–258**

- `concatenate.py`, [413–414](#)
- `concentric_circles.py`, [197–199](#)
- `contact_manager.py`, [524–532](#)
- `contact.py`, [523](#)
- `count_Ts.py`, [410](#)
- `cups_to_ounces.py`, [228–229](#)
- `customer.py`, [540](#)
- `delete_coffee_record.py`, [326–327](#)
- `dice.py`, [245](#)
- `display_file.py`, [328–329](#)
- `display_file2.py`, [329–330](#)
- `division.py`, [324–325](#)
- `dollar_display.py`, [70–71](#)
- `draw_arc.py`, [638–639](#)
- `draw_circles.py`, [270–271](#)
- `draw_line.py`, [630–631](#)
- `draw_lines.py`, [272–273](#)
- `draw_multi_lines.py`, [632–633](#)
- `draw_ovals.py`, [636–637](#)
- `draw_piechart.py`, [641–643](#)
- `draw_polygon.py`, [643–644](#)
- `draw_square.py`, [634](#)
- `draw_squares.py`, [269–270](#)
- `draw_text.py`, [645–646](#)
- `drop_lowest_score.py`, [370–373](#)

- `empty_window1.py`, **600–601**
- `empty_window2.py`, **601–602**
- `endless_recursion.py`, **577–578**
- `fibonacci.py`, **586–587**
- `file_read.py`, **294–295**
- `file_write.py`, **293–294**
- `font_demo.py`, **649–650**
- `formatting.py`, **68–69**
- `frame_demo.py`, **606–607**
- `function_demo.py`, **213–214**
- `gcd.py`, **587–588**
- `generate_login.py`, **417–418**
- `geometry.py`, **266–268**
- `global1.py`, **235–236**
- `global2.py`, **236**
- `grader.py`, **130–131**
- `graphics_mod_demo.py`, **274–275**
- `gross_pay.py`, **229**
- `gross_pay1.py`, **326–327**
- `gross_pay2.py`, **327–328**
- `gross_pay3.py`, **333–334**
- `hello_world.py`, **602–603**
- `hello_world2.py`, **604**
- `hello_world3.py`, **604–605**
- `hit_the_target.py`, **145–148**

- `hypotenuse.py`, **262–263**
- `index_list.py`, **358**
- `infinite.py`, **167**
- `in_list.py`, **354–355**
- `input.py`, **52–53**
- `keyword_args.py`, **233–234**
- `keywordstringargs.py`, **234**
- `kilo_converter.py`, **612–614**
- `kilo_converter2.py`, **615–617**
- `line_graph1.py`, **384–385**
- `line_graph2.py`, **385–386**
- `list_append.py`, **356–357**
- `loan_qualifier.py`, **127–128**
- `loan_qualifier2.py`, **136–137**
- `loan_qualifier3.py`, **137–138**
- `login.py`, **416–417, 425–427**
- `modify_coffee_records.py`, **320–321**
- `multiple_args.py`, **229–230**
- `my_graphics.py`, **273–274**
- `no_formatting.py`, **68–69**
- `orion.py`, **96–99**
- `pass_arg.py`, **226**
- `password.py`, **121–122**
- `pickle_cellphone.py`, **520–521**
- `pickle_objects.py`, **476–478**

- `polymorphism_demo.py`, 569–570
- `polymorphism_demo2.py`, 571–572
- `property_tax.py`, 183–184
- `quit_button.py`, 610–611
- `radiobutton_demo.py`, 623–624
- `random_numbers.py`, 242, 379
- `random_numbers2.py`, 242–243
- `read_emp_records.py`, 314–315
- `read_list.py`, 374–375
- `read_number_list.py`, 376
- `read_number.py`, 303
- `read_running_times.py`, 310–311
- `read_sales.py`, 307
- `read_sales2.py`, 308
- `rectangle.py`, 266
- `rectangular_pattern.py`, 238–239
- `repetition_operator.py`, 428–429
- `retail_no_validation.py`, 188
- `retail_with_validation.py`, 189
- `return_list.py`, 368–369
- `sale_price.py`, 55–56, 253–254
- `sales_list.py`, 348–349
- `sales_report1.py`, 330
- `sales_report2.py`, 332–333
- `sales_report3.py`, 334

- `sales_report4.py`, **335–336**
- `save_emp_records.py`, **316–317**
- `save_running_times.py`, **309–310**
- `search_coffee_records.py`, **318**
- `servicequote.py`, **542–543**
- `sets.py`, **471–473**
- `show_coffee_records.py`, **316–317**
- `simple_loop1.py`, **169**
- `simple_loop2.py`, **170**
- `simple_loop3.py`, **170**
- `simple_loop4.py`, **171**
- `simple_math.py`, **54–55**
- `sort_names.py`, **124**
- `speed_converter.py`, **175–176**
- `spiral_circles.py`, **199**
- `spiral_lines.py`, **200**
- `split_date.py`, **430**
- `square_root.py`, **262**
- `squares.py`, **173**
- `stair_step_pattern.py`, **196**
- `string_args.py`, **231**
- `string_input.py`, **50**
- `string_split.py`, **429**
- `string_test.py`, **421**
- `string_variable.py`, **48**

- `strip_newline.py`, [303–304](#)
- `sum_numbers.py`, [180](#)
- `temperature.py`, [166–167](#)
- `test_average.py`, [116–117](#)
- `test_averages.py`, [620–622](#)
- `test_score_average.py`, [59](#)
- `test_score_averages.py`, [192–193](#)
- `time_converter.py`, [60–61](#)
- `total_ages.py`, [251](#)
- `total_function.py`, [368](#)
- `total_list.py`, [366](#)
- `towers_of_hanoi.py`, [590–591](#)
- `triangle_pattern.py`, [195](#)
- `two_functions.py`, [214–215](#)
- `unpickle_cellphone.py`, [521–522](#)
- `unpickle_objects.py`, [478–480](#)
- `user_squares1.py`, [176–177](#)
- `user_squares2.py`, [177–178](#)
- `validate_password.py`, [427](#)
- `variable_demo.py`, [42](#)
- `variable_demo2.py`, [42](#)
- `variable_demo3.py`, [45](#)
- `variable_demo4.py`, [45–46](#)
- `vehicles.py`, [553–554](#), [555](#), [557](#), [558](#)
- `writelines.py`, [373](#)

- `write_list.py`, [374](#)
- `write_names.py`, [298](#)
- `write_number_list.py`, [375](#)
- `write_numbers.py`, [301](#)
- `write_sales.py`, [304–305](#)
- `wrong_type.py`, [570](#)
- Pseudocode, [34](#)
- Pseudorandom numbers, [248–249](#)
- Public methods, [492](#)
- Python
 - defined, [17](#)
 - downloading, [659](#)
 - execution by, [665](#)
 - IDLE, [23–24](#), [663](#)
 - installing, [20](#), [659–661](#)
 - interactive mode, [21–22](#)
 - interpreter, [19](#), [21](#)
 - key words, [17](#)
 - in learning programming, [1–2](#)
 - script mode, [22–23](#)
 - Shell window, [663](#), [664](#), [669](#)
- Python programs
 - running, [23](#)
 - writing in IDLE editor, [665–666](#)
 - writing in script mode, [22–23](#)

Q

- Quit button, [610–613](#)

R

- `radians()` function, [263](#)
- Radio buttons
 - defined, [622](#)
 - group of, [622](#)
 - illustrated, [622](#)
 - use example, [623–625](#)
 - uses, [622](#)
- `Radiobutton` widget
 - callback functions with, [625](#)
 - creation and use example, [622–625](#)
 - defined, [600](#)
 - as mutually exclusive, [623](#)
- RAM. see [Random access memory \(RAM\)](#)
- `randint` function
 - calling, [241](#)
 - defined, [241](#)
 - in interactive mode, [244](#)
 - in math expression, [246](#)
 - random number generation, [242–244](#)
 - return value, [241–242](#)
 - use examples, [243–244](#)
- Random access files, [290](#)
- Random access memory (RAM), [5](#)

- `random` module, [241](#), [247](#), [248–249](#)
- Random numbers
 - displaying, [243](#)
 - example uses, [240–241](#)
 - generating, [241–244](#)
 - interactive mode, [244](#)
 - pseudo, [248–249](#)
 - to represent other values, [246–247](#)
 - seeds, [248–249](#)
 - using, [244–245](#)
- `range` function
 - arguments passed to, [171–172](#)
 - default, [171](#)
 - generation, [171](#)
 - iterable object, converting to list, [170–171](#), [344–345](#)
 - with `for` loop, [170–172](#)
- Raw string, [296](#)
- Read, priming, [186](#), [306](#)
- Reading
 - data from files, [289](#), [294–297](#)
 - files with loops, [305–307](#)
 - input from keyboard, [49–51](#)
 - with `for` loop, [307–308](#)
 - numbers from text files, [302](#)
 - numbers with input function, [51–53](#)
 - numeric data, [301–304](#)
 - records, [313](#)
 - strings, and stripping newline, [298–300](#)

- `readline` method
 - defined, [295](#)
 - empty string return, [306](#)
 - example use, [295–296](#)
 - reading strings from files with, [303](#)
 - return, [295](#), [297](#)
- `readlines` method, [374](#)
- `read` method, [294–295](#)
- Read position
 - advanced to the end of file, [297](#)
 - advance to next line, [296–297](#)
 - defined, [296](#)
 - initial, [296](#)
- Records
 - adding, [315–317](#)
 - defined, [311](#)
 - deleting, [322–324](#)
 - displaying, [315–317](#)
 - fields, [312](#)
 - files, [312](#)
 - modifying, [319–321](#)
 - processing, [311–324](#)
 - programs storing, [315](#)
 - searching, [317–319](#)
 - writing to sequential access files, [312](#)
- Recursion
 - depth of, [580](#)
 - direct, [584](#)

- in factorial calculation, [581–583](#)
- indirect, [584](#)
- list elements with, [584–585](#)
- loops vs., [580–581](#), [591](#)
- problem solving with, [580–584](#)
- stopping of, [583](#)
- Recursive algorithms
 - designing, [581](#)
 - efficiency, [580](#)
 - examples of, [584–591](#)
 - Fibonacci series, [585–587](#)
 - greatest common divisor, [587–588](#)
 - recursive call, [583](#)
 - summing range of list elements, [584–585](#)
 - Towers of Hanoi, [588–591](#)
- Recursive case, [581](#)
- Recursive functions
 - call, [580](#)
 - defined, [577](#)
 - efficiency, [591](#)
 - example, [577–580](#)
 - functioning of, [581](#)
 - overhead, [580](#)
- Relational operators
 - defined, [111](#)
 - in string comparisons, [123–124](#)
 - types of, [112](#)
- Remainder (%) operator

- defined, [54](#), [60](#)
 - precedence, [57](#)
 - use example, [60–61](#)
- `remove` method, [356](#), [360–361](#), [465–466](#)
- Repetition operator (*)
 - defined, [345](#)
 - general format, [345](#), [428](#)
 - for lists, [345](#)
 - for strings, [428–429](#)
 - use example, [428–429](#)
- Repetition structures. see also [Loops](#)
 - defined, [159](#), [160](#)
 - example, [159–160](#)
 - flowchart, [164](#)
 - introduction to, [159–160](#)
- `replace` method, [423](#), [424](#)
- Reserved words. see [Key words](#)
- Returning
 - Boolean values, [259–260](#)
 - dictionary values, [450–452](#)
 - key-value pairs, [451](#)
 - lists from functions, [368–370](#)
 - multiple values, [260–241](#)
 - strings, [258](#)
- `return` statement
 - defined, [250](#)
 - form, [250](#)
 - result variable and, [252](#), [261](#)

- using, [252](#)
- `reverse` method, [356](#), [361](#)
- Rounding, [56](#)
- `rstrip()` method, [299](#), [422](#)
- Ruby, [17](#)
- Running programs, [668–669](#)
- Running totals
 - calculating, [179–182](#)
 - defined, [179](#)
 - elements for calculating, [179](#)
 - example, [180](#)
 - logic for calculating, [179](#)

S

- Samples, [12](#)
- Saving programs, [668](#)
- `SavingsAccount` class, [562](#), [563](#)
- `Scale` widget, [600](#)
- Scientific notation, [70](#)
- Scope
 - defined, [223](#)
 - local variables and, [223–225](#)
 - parameter variable, [227](#)
- Script mode, running programs in, [23](#)
- `Scrollbar` widget, [600](#)
- Searching
 - lists, [354–355](#)
 - records, [317–319](#)
 - strings, [423–424](#)
- Secondary storage
 - defined, [5](#)
 - devices, [5–6](#)
- Seeds, random number, [248–249](#)
- Seed value, [248](#)
- Selection structures. see [Decision structures](#)
- Sentinels
 - defined, [182](#), [183](#)

- using, [183–184](#)
- values, [182–183](#)
- Separators
 - comma, [70–71](#)
 - item, [66–67](#)
 - `split` method, [429–430](#)
- Sequences. see also [Lists](#)
 - accepting as argument, [361–362](#)
 - defined, [343](#)
 - length, returning, [347](#)
 - tuples, [380–382](#)
- Sequence structures
 - with decision structure, [125](#)
 - defined, [109](#)
 - example, [109](#)
 - nested inside decision structure, [126](#)
 - structure, [125](#), [126](#)
 - use in programming, [109–110](#)
- Serializing objects
 - defined, [474](#)
 - example, [476–478](#), [520–522](#)
 - `pickle` module, [474–475](#)
 - unserializing, [478–480](#), [521–522](#)
- `ServiceQuote` class, [541–543](#)
- `set` function, [463](#)
- `set` method, [618](#), [625](#)
- Sets
 - adding elements, [464–465](#)

- creating, [463](#)
- defined, [462](#)
- difference of, [468–469](#)
- duplicate elements, [463](#)
- elements, as unique, [462](#)
- elements, number of, [464](#)
- elements, removing, [465–466](#)
- intersection of, [468](#)
- `for` loop iteration over, [466](#)
- operations, [471–473](#)
- subsets, [470](#)
- supersets, [470](#)
- symmetric difference of, [469](#)
- union of, [467](#)
- unordered, [462](#)
- values, testing, [466–467](#)
- Settings. see [Mutator methods](#)
- `Showinfo` function, [608](#)
- `sin()` function, [263](#)
- Single alternative decision structures, [110](#)
- Slices
 - defined, [351](#)
 - example use, [351–353](#)
 - general format, [351](#)
 - invalid indexes and, [353](#)
 - list, [351–354](#)
 - start and end index, [352](#)
 - string, [415–416](#)

- Software
 - application, [7](#)
 - defined, [1](#)
 - developers. see [Programmers](#)
 - development tools, [7](#)
 - operating system, [6](#)
 - requirements, [33](#)
 - system, [6–7](#)
 - types of, [6](#)
 - utility programs, [6](#)
- Sorting, item list, [359–360](#)
- `sort` method, [356](#), [359–360](#)
- Source code. see [Code](#)
- Specialization, [551–552](#)
- `split` method, [429](#)
- Splitting strings, [429–430](#)
- `sqrt()` function, [263](#)
- `startswith` method, [423](#), [424](#)
- Statements
 - `for`, [168](#), [172](#), [174](#)
 - assignment, [40–43](#)
 - in blocks, [111](#), [213](#)
 - breaking into multiple lines, [64–65](#)
 - converting math formulas to, [61–63](#)
 - defined, [18](#)
 - `del`, [361](#)
 - `if`, [109–117](#)

- `if-elif-else`, [125–133](#)
- `if-else`, [118–121](#)
- `import`, [239–240](#)
- `return`, [250–252](#), [261](#)
- `try/except`, [324](#), [326–327](#), [332–333](#), [335](#), [336](#)
- Step values, [172](#)
- `str` data type, [47–48](#)
- `str` function
 - defined, [304](#)
 - example use, [301](#)
- String comparisons, [122–125](#), [422–423](#), [441](#), [442](#)
- String concatenation
 - defined, [68](#), [412](#)
 - example, [68](#)
 - interactive sessions, [412](#)
 - newline to, [297–298](#)
 - with + operator, [68](#)
 - with += operator, [413](#)
- String literals
 - defined, [37](#)
 - enclosing in triple quotes, [38](#)
- Strings
 - characters, accessing, [408–412](#)
 - comparing, [122–126](#)
 - defined, [37](#)
 - extracting characters from, [416–418](#)
 - as immutable, [413–414](#)

- indexes, [410–411](#)
- iterating with `for` loop, [408–410](#)
- list of, [344](#)
- method call format, [420](#)
- modification methods, [422–423](#)
- operations, [407–414](#)
- raw, [292](#)
- reading as input, [407](#)
- repetition operator (*) with, [428–429](#)
- returning, [258](#)
- search and replace methods, [423–424](#)
- slicing, [415–416](#)
- space character at end, [51](#)
- splitting, [429–430](#)
- storing with str data type, [47–48](#)
- stripping newline from, [298–300](#)
- testing, [419](#)
- testing methods, [420–421](#)
- writing as output, [407](#)
- written to files, [294](#)
- `StringVar` object, [614–615](#)
- `strip()` method, [422](#)
- Stripping newline from string, [298–300](#)
- Structure charts. see [Hierarchy charts](#)
- Subclasses
 - defined, [552](#)
 - as derived classes, [552](#)
 - method override, [566](#)

- polymorphism, [566–572](#)
 - writing, [553](#)
- Subscripts, [378](#)
- Subsets, [470](#)
- Subtraction (–) operator
 - defined, [54](#)
 - precedence, [57](#)
- Superclasses
 - as base classes, [552](#)
 - defined, [552](#)
 - in UML diagrams, [560–561](#)
 - writing, [553](#)
- Supersets, [470](#)
- SUV class, [558–559](#)
- Symbols, flowchart, [34](#)
- `symmetric_difference` method, [469](#)
- Symmetric difference of sets, [469](#)
- Syntax
 - defined, [18](#)
 - human language, [19](#)
- Syntax errors
 - correcting, [32](#)
 - defined, [19](#)
 - reporting, [669](#)
 - running programs and, [669](#)
- System software, [6–7](#)

T

- `tan()` function, [263](#)
- Target variables
 - defined, [170](#)
 - inside for loops, [172–174](#)
 - use example, [173](#)
- Terminal symbols, [34](#)
- Testing
 - dictionary values, [441–442](#)
 - programs, [32–33](#)
 - set values, [466–467](#)
 - string methods, [420–421](#)
 - strings, [419](#)
- Text
 - drawing with `Canvas` widget, [645–650](#)
 - displaying with `Label` widget, [602–605](#)
 - editing, [665–666](#)
 - font setting, [648–650](#)
- Text files
 - defined, [289](#)
 - reading numbers from, [302](#)
- `Text` widget, [600](#)
- third-party modules, [683](#)
- `Tkinter` module

- defined, [599](#)
- programs using, [600](#)
- widgets, [600](#)
- Top-down design
 - defined, [218](#)
 - process, [218–219](#)
- `Toplevel` widget, [600](#)
- Totals
 - list values, [366](#)
 - running, [179–182](#)
- Towers of Hanoi. see also [Recursive algorithms](#)
 - base case, [590](#)
 - defined, [588](#)
 - illustrated, [588](#)
 - problem solution, [589–590](#)
 - program code, [590–591](#)
 - steps for moving pegs, [589](#)
- Tracebacks, [325](#), [329](#)
- `Truck` class, [557–558](#)
- Truncation, in integer division, [56](#)
- Truth tables
 - `not` operator, [135](#)
 - `and` operator, [134](#)
 - `or` operator, [134](#)
- `try/except` statement
 - `else` clause, [335–336](#)
 - execution of, [327](#)

- `finally` clause, 336
- handling exceptions with, 328, 330–331
- one `except` clause, 332–333
- use example, 327–328
- Tuples, 380–382
- Turtle module, 75
- `turtle.bgcolor()` function, 142
- `turtle.bgcolor(color)` function, 83
- `turtle.circle(radius)` function, 81
- `turtle.clear()` function, 84
- `turtle.clearscreen()` function, 84
- `turtle.dot()` function, 82
- `turtle.down()` function, 91
- `turtle.fillcolor()` function, 142
- `turtle.forward(n)` function, 75
- `turtle.heading()` function, 141
- `turtle.heading()` function, 80
- `turtle.hideturtle()` function, 87
- `turtle.isdown()` function, 141
- `turtle.isvisible()` function, 141
- `turtle.pencolor()` function, 142
- `turtle.pencolor(color)` function, 83
- `turtle.pendown()` function, 81
- `turtle.pensize()` function, 142–143
- `turtle.pensize(width)` function, 82

- `turtle.penup()` function, [81](#)
- `turtle.pos()` function, [86](#)
- `turtle.reset()` function, [84](#)
- `turtle.setheading(angle)` function, [79–80](#)
- `turtle.setup(width, height)` function, [84](#)
- `turtle.showturtle()` function, [75](#)
- `turtle.speed()` function, [143](#)
- `turtle.speed(speed)` function, [86](#)
- `turtle.write(text)` function, [87–88](#)
- Turtles/turtle graphics, [74–75](#)
 - animation speed, controlling, [86](#)
 - animation speed, determination of, [143](#)
 - background color, changing, [83](#)
 - circles, drawing with, [81–82](#)
 - current colors, determination of, [142](#)
 - current heading, [80](#)
 - current position, [86](#)
 - displaying text in window, [87–88](#)
 - dots, drawing with, [81–82](#)
 - drawing color, changing, [83](#)
 - filling shapes, [88–90](#)
 - functions, [268–275](#)
 - heading, determination of, [141](#)
 - hiding, [87](#)
 - hit the target game, [143–148](#)
 - lines drawing with, [75](#)
 - locations, determination of, [140–141](#)

- loops in, [197–200](#)
- moving to specific location, [84–85](#)
- pen, changing size, [82](#)
- pen, up and down movement, [80–81](#)
- pen is down, determination of, [141](#)
- pen size, determination of, [142–143](#)
- setting heading to specific angle, [79–80](#)
- state of turtle, determination of, [140–148](#)
- turning, [75](#), [77–79](#)
- using, [91–98](#)
- visibility, determination of, [141](#)
- window, resetting, [84](#)
- window, size of, [84](#)
- Two-dimensional lists, [376–380](#). see also [Lists](#)
- Two's complement, [12](#)
- `type` function, [47](#)

U

- UML diagrams
 - `Customer` class, [539](#)
 - inheritance in, [560–561](#)
 - layout, [533](#)
 - `ServiceQuote` class, [542](#)
- Unicode, [11](#)
- Unified Modeling Language (UML), [532](#)
- `union` method, [467](#)
- Union of sets, [467](#)
- `update` method, [464](#)
- `upper()` method, [422](#)
- USB drives, [5–6](#)
- User interfaces
 - command line, [597–599](#)
 - defined, [597](#)
 - graphical user (GUIs), [597–657](#)
- Utility programs, [6](#)

V

- Validation
 - code, Boolean functions in, [260](#)
 - input, [187](#)
 - password characters, [424–427](#)
 - `ValueError`, [327–328](#), [332](#), [333](#), [334](#), [336](#), [360](#)
- Value-returning functions
 - benefits, [252](#)
 - defined, [239](#)
 - general format, [250](#)
 - how to use, [252–254](#)
 - and IPO charts, [254–255](#)
 - `randint` and, [241–244](#)
 - random number generation, [240–249](#)
 - `randrange` and, [247–248](#)
 - returning multiple values, [260–261](#)
 - `return` statement, [250–252](#)
 - for simplifying mathematical expressions, [252–254](#)
 - values, [239](#), [252](#)
 - void functions and, [211](#)
 - writing, [250–261](#)
- Values
 - Boolean, [259–260](#)
 - data attribute, [492](#)

- dictionaries, [441–442](#)
- global constants, [237](#)
- list, averaging, [366–367](#)
- list, totaling, [366](#)
- multiple, returning, [260–261](#)
- random numbers to represent, [246–247](#)
- range of, [138](#)
- seed, [248](#)
- value-returning functions, [239](#), [252](#)
- `values` method, [451–452](#)
- Variables
 - Boolean, [139–140](#)
 - defined, [40](#), [45](#)
 - global, [235–237](#)
 - local, [223–225](#), [232](#), [261](#), [580](#)
 - `math` module, [263](#)
 - names, sample, [44](#)
 - naming rules, [43–44](#)
 - reassignment, [45–46](#)
 - reassignment to different type, [48](#)
 - scope, [223–224](#)
 - statements, [40–43](#)
 - target, [172](#)
- Visual Basic, [17](#)

W

- `while` loops. see also [Loops](#)
- Widgets
 - arrangement of, [608](#)
 - `Button`, [600](#), [608–611](#)
 - `Canvas`, [600](#)
 - `Checkbutton`, [600](#), [626–628](#)
 - defined, [600](#)
 - `Entry`, [600](#), [611–614](#)
 - `Frame`, [600](#), [607](#)
 - with frames, organizing, [605–608](#)
 - `Label`, [600](#), [603–604](#), [607](#), [614–618](#)
 - `Listbox`, [600](#)
 - list of, [619](#)
 - `Menu`, [600](#)
 - `Menubutton`, [600](#)
 - `Message`, [600](#)
 - `Radiobutton`, [600](#), [622–625](#)
 - `Scale`, [600](#)
 - `Scrollbar`, [600](#)
 - `Text`, [600](#)
 - `tkinter` module, [600](#)
 - `Toplevel`, [600](#)

- wildcard `import` statement, [680](#)
- `writelines` method, [373](#)
- `write` method
 - defined, [292](#)
 - format for calling, [292–293](#)
- Writing
 - code, [32](#)
 - data to files, [292](#), [296–298](#)
 - numeric data, [305–307](#)
 - programs in IDLE editor, [665–666](#)
 - records, [316](#)

Z

- `ZeroDivisionError` exception, [336](#)

Credits

Cover Image © Westend61 GmbH/Alamy Stock Photo

Figure 1.2  a pg. 3 © iko/Shutterstock

Figure 1.2  b pg. 3 © Nikita Rogul/Shutterstock

Figure 1.2  c pg. 3 © Feng Yu/Shutterstock

Figure 1.2  d pg. 3 © Chiyacat/Shutterstock

Figure 1.2  e pg. 3 © Eikostas/Shutterstock

Figure 1.2  f pg. 3 © tkemot/Shutterstock

Figure 1.2  g pg. 3 © Vitaly Korovin/Shutterstock

Figure 1.2  h pg. 3 © Lusoimages/Shutterstock

Figure 1.2  i pg. 3 © jocic/Shutterstock

Figure 1.2  j pg. 3 © Best Pictures/Shutterstock

Figure 1.2  k pg. 3 © Peter Guess/Shutterstock

Figure 1.2  pg. 3 © Aquila/Shutterstock

Figure 1.2 m pg. 3 © Andre Nitsievsky/Shutterstock

Figure 1.3  pg. 4 Courtesy of US Army Historic Computer Images

Figure 1.4  pg. 4 © Creativa/Shutterstock

Figure 1.5  pg. 5 © Garsya/Shutterstock

Chapter 1  Microsoft screenshots - SEE MICROSOFT AGREEMENT FOR FULL CREDIT.

Chapter 6  Microsoft screenshots - SEE MICROSOFT AGREEMENT FOR FULL CREDIT.

Chapter 8  Microsoft screenshots - SEE MICROSOFT AGREEMENT FOR FULL CREDIT.

Chapter 9  Microsoft screenshots - SEE MICROSOFT AGREEMENT FOR FULL CREDIT.

Chapter 13  Microsoft screenshots - SEE MICROSOFT AGREEMENT FOR FULL CREDIT.

MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THE INFORMATION CONTAINED IN THE DOCUMENTS AND RELATED

GRAPHICS PUBLISHED AS PART OF THE SERVICES FOR ANY PURPOSE. ALL SUCH DOCUMENTS AND RELATED GRAPHICS ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND. MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS HEREBY DISCLAIM ALL WARRANTIES AND CONDITIONS WITH REGARD TO THIS INFORMATION, INCLUDING ALL WARRANTIES AND CONDITIONS OF MERCHANTABILITY, WHETHER EXPRESS, IMPLIED OR STATUTORY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF INFORMATION AVAILABLE FROM THE SERVICES. THE DOCUMENTS AND RELATED GRAPHICS CONTAINED HEREIN COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN. MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED HEREIN AT ANY TIME. PARTIAL SCREEN SHOTS MAY BE VIEWED IN FULL WITHIN THE SOFTWARE VERSION SPECIFIED.

Contents

1. **Starting Out with Python®**
2. **Starting Out with Python®**
3. **Contents in a Glance**
4. **Contents**
5. **Location of Videonotes in the Text**
6. **Preface**
 - A. **Control Structures First, Then Classes**
 - B. **Changes in the Fourth Edition**
 - C. **Brief Overview of Each Chapter**
 - D. **Organization of the Text**
 - E. **Features of the Text**
 - F. **Supplements**
 - G. **Reviewers of Previous Editions**
 - H. **About the Author**
7. **Chapter 1 Introduction to Computers and Programming**
 - A. **Topics**
 - B. **1.1 Introduction**
 - C. **1.2 Hardware and Software**
 1. **Hardware**
 2. **The CPU**
 3. **Main Memory**
 4. **Secondary Storage Devices**
 5. **Input Devices**

6. **Output Devices**
7. **Software**
8. **System Software**
9. **Application Software**
 - a. **Checkpoint**

D. 1.3 How Computers Store Data

1. **Storing Numbers**
2. **Storing Characters**
3. **Advanced Number Storage**
4. **Other Types of Data**
 - a. **Checkpoint**

E. 1.4 How a Program Works

1. **From Machine Language to Assembly Language**
2. **High-Level Languages**
3. **Key Words, Operators, and Syntax: An Overview**
4. **Compilers and Interpreters**
 - a. **Checkpoint**

F. 1.5 Using Python

1. **Installing Python**
2. **The Python Interpreter**
3. **Interactive Mode**
Writing Python Programs and Running Them in Script
4. **Mode**
5. **The IDLE Programming Environment**

G. Review Questions

- 1. Multiple Choice**
- 2. True or False**
- 3. Short Answer**
- 4. Exercises**

8. Chapter 2 Input, Processing, and Output

A. Topics

B. 2.1 Designing a Program

- 1. The Program Development Cycle**
- 2. More About the Design Process**
- 3. Understand the Task That the Program Is to Perform
Determine the Steps That Must Be Taken to Perform the**
- 4. Task**
- 5. Pseudocode**
- 6. Flowcharts**
 - a. Checkpoint**

C. 2.2 Input, Processing, and Output

D. 2.3 Displaying Output with the `print` Function

- 1. Strings and String Literals**
 - a. Checkpoint**

E. 2.4 Comments

F. 2.5 Variables

- 1. Creating Variables with Assignment Statements**
- 2. Variable Naming Rules**
- 3. Displaying Multiple Items with the `print` Function**

4. **Variable Reassignment**
 5. **Numeric Data Types and Literals**
 6. **Storing Strings with the `str` Data Type**
 7. **Reassigning a Variable to a Different Type**
 - a. **Checkpoint**
- G. **2.6 Reading Input from the Keyboard**
1. **Reading Numbers with the `input` Function**
 - a. **Checkpoint**
- H. **2.7 Performing Calculations**
1. **Floating-Point and Integer Division**
 2. **Operator Precedence**
 3. **Grouping with Parentheses**
 4. **The Exponent Operator**
 5. **The Remainder Operator**
 6. **Converting Math Formulas to Programming Statements**
 7. **Mixed-Type Expressions and Data Type Conversion**
 8. **Breaking Long Statements into Multiple Lines**
 - a. **Checkpoint**
- I. **2.8 More About Data Output**
1. **Suppressing the `print` Function's Ending Newline**
 2. **Specifying an Item Separator**
 3. **Escape Characters**
 4. **Displaying Multiple Items with the `+` Operator**
 5. **Formatting Numbers**
 6. **Formatting in Scientific Notation**

7. **Inserting Comma Separators**
8. **Specifying a Minimum Field Width**
9. **Formatting a Floating-Point Number as a Percentage**
10. **Formatting Integers**
 - a. **Checkpoint**

J. **2.9 Named Constants**

1. **Checkpoint**

K. **2.10 Introduction to Turtle Graphics**

1. **Drawing Lines with the Turtle**
2. **Turning the Turtle**
3. **Setting the Turtle's Heading to a Specific Angle**
4. **Getting the Turtle's Current Heading**
5. **Moving the Pen Up and Down**
6. **Drawing Circles and Dots**
7. **Changing the Pen Size**
8. **Changing the Drawing Color**
9. **Changing the Background Color**
10. **Resetting the Screen**
11. **Specifying the Size of the Graphics Window**
12. **Moving the Turtle to a Specific Location**
13. **Getting the Turtle's Current Position**
14. **Controlling the Turtle's Animation Speed**
15. **Hiding the Turtle**
16. **Displaying Text in the Graphics Window**
17. **Filling Shapes**
18. **Using `turtle.done()` to Keep the Graphics Window Open**

- a. Checkpoint

- L. Review Questions

- 1. Multiple Choice
- 2. True or False
- 3. Short Answer
- 4. Algorithm Workbench

- M. Programming Exercises

- 9. Chapter 3 Decision Structures and Boolean Logic

- A. Topics

- B. 3.1 The `if` Statement

- 1. Boolean Expressions and Relational Operators
 - a. The `>=` and `<=` Operators
 - b. The `==` Operator
 - c. The `!=` Operator
- 2. Putting It All Together
 - a. Checkpoint

- C. 3.2 The `if-else` Statement

- 1. Indentation in the if-else Statement
 - a. Checkpoint

- D. 3.3 Comparing Strings

- 1. Other String Comparisons
 - a. Checkpoint

3.4 Nested Decision Structures and the `if-elif-else`

E. Statement

1. Testing a Series of Conditions
2. The `if-elif-else` Statement
 - a. Checkpoint

F. 3.5 Logical Operators

1. The `and` Operator
2. The `or` Operator
3. Short-Circuit Evaluation
4. The `not` Operator
5. The Loan Qualifier Program Revisited
6. Yet Another Loan Qualifier Program
7. Checking Numeric Ranges with Logical Operators
 - a. Checkpoint

G. 3.6 Boolean Variables

1. Checkpoint

H. 3.7 Turtle Graphics: Determining the State of the Turtle

1. Determining the Turtle's Location
2. Determining the Turtle's Heading
3. Determining Whether the Pen Is Down
4. Determining Whether the Turtle Is Visible
5. Determining the Current Colors
6. Determining the Pen Size
7. Determining the Turtle's Animation Speed
 - a. Checkpoint

- I. **Review Questions**
 - 1. **Multiple Choice**
 - 2. **True or False**
 - 3. **Short Answer**
 - 4. **Algorithm Workbench**

- J. **Programming Exercises**

- 0. **Chapter 4 Repetition Structures**

- A. **Topics**

- B. **4.1 Introduction to Repetition Structures**

- 1. **Condition-Controlled and Count-Controlled Loops**
 - a. **Checkpoint**

- C. **4.2 The `while` Loop: A Condition-Controlled Loop**

- 1. **The `while` Loop Is a Pretest Loop**
 - 2. **Infinite Loops**
 - a. **Checkpoint**

- D. **4.3 The `for` Loop: A Count-Controlled Loop**

- 1. **Using the `range` Function with the `for` Loop**
 - 2. **Using the Target Variable Inside the Loop**
 - 3. **Letting the User Control the Loop Iterations**
 - Generating an Iterable Sequence that Ranges from**
 - 4. **Highest to Lowest**
 - a. **Checkpoint**

- E. **4.4 Calculating a Running Total**

1. **The Augmented Assignment Operators**
 - a. **Checkpoint**
- F. **4.5 Sentinels**
 1. **Checkpoint**
- G. **4.6 Input Validation Loops**
 1. **Checkpoint**
- H. **4.7 Nested Loops**
- I. **4.8 Turtle Graphics: Using Loops to Draw Designs**
- J. **Review Questions**
 1. **Multiple Choice**
 2. **True or False**
 3. **Short Answer**
 4. **Algorithm Workbench**
- K. **Programming Exercises**
1. **Chapter 5 Functions**
 - A. **Topics**
 - B. **5.1 Introduction to Functions**
 1. **Benefits of Modularizing a Program with Functions**
 - a. **Simpler Code**
 - b. **Code Reuse**
 - c. **Better Testing**
 - d. **Faster Development**
 - e. **Easier Facilitation of Teamwork**

- 2. **Void Functions and Value-Returning Functions**
 - a. **Checkpoint**
- C. **5.2 Defining and Calling a Void Function**
 - 1. **Function Names**
 - 2. **Defining and Calling a Function**
 - a. **Calling a Function**
 - 3. **Indentation in Python**
 - a. **Checkpoint**
- D. **5.3 Designing a Program to Use Functions**
 - 1. **Flowcharting a Program with Functions**
 - 2. **Top-Down Design**
 - 3. **Hierarchy Charts**
 - 4. **Pausing Execution Until the User Presses Enter**
- E. **5.4 Local Variables**
 - 1. **Scope and Local Variables**
 - a. **Checkpoint**
- F. **5.5 Passing Arguments to Functions**
 - 1. **Parameter Variable Scope**
 - 2. **Passing Multiple Arguments**
 - 3. **Making Changes to Parameters**
 - 4. **Keyword Arguments**
 - Mixing Keyword Arguments with Positional**
 - a. **Arguments**

- a. Checkpoint

- G. **5.6 Global Variables and Global Constants**

- 1. **Global Constants**

- a. Checkpoint

- 5.7 Introduction to Value-Returning Functions: Generating**

- H. **Random Numbers**

- 1. **Standard Library Functions and the `import` Statement**

- 2. **Generating Random Numbers**

- Experimenting with Random Numbers in Interactive

- 3. **Mode**

- 4. **The `randrange`, `random`, and `uniform` Functions**

- 5. **Random Number Seeds**

- a. Checkpoint

- I. **5.8 Writing Your Own Value-Returning Functions**

- 1. **Making the Most of the `return` Statement**

- 2. **How to Use Value-Returning Functions**

- 3. **Using IPO Charts**

- 4. **Returning Strings**

- 5. **Returning Boolean Values**

- a. **Using Boolean Functions in Validation Code**

- 6. **Returning Multiple Values**

- a. Checkpoint

- J. **5.9 The `math` Module**

1. **The `math.pi` and `math.e` Values**
 - a. **Checkpoint**
 - K. **5.10 Storing Functions in Modules**
 1. **Menu-Driven Programs**
 - L. **5.11 Turtle Graphics: Modularizing Code with Functions**
 1. **Storing Your Graphics Functions in a Module**
 - M. **Review Questions**
 1. **Multiple Choice**
 2. **True or False**
 3. **Short Answer**
 4. **Algorithm Workbench**
 - N. **Programming Exercises**
2. **Chapter 6 Files and Exceptions**
 - A. **Topics**
 - B. **6.1 Introduction to File Input and Output**
 1. **Types of Files**
 2. **File Access Methods**
 3. **Filenames and File Objects**
 4. **Opening a File**
 5. **Specifying the Location of a File**
 6. **Writing Data to a File**
 7. **Reading Data From a File**
 8. **Concatenating a Newline to a String**

9. Reading a String and Stripping the Newline from It
10. Appending Data to an Existing File
11. Writing and Reading Numeric Data
 - a. Checkpoint

C. **6.2 Using Loops to Process Files**

Reading a File with a Loop and Detecting the End of the

1. File
2. Using Python's `for` Loop to Read Lines
 - a. Checkpoint

D. **6.3 Processing Records**

1. Checkpoint

E. **6.4 Exceptions**

1. Handling Multiple Exceptions
2. Using One `except` Clause to Catch All Exceptions
3. Displaying an Exception's Default Error Message
4. The `else` Clause
5. The `finally` Clause
6. What If an Exception Is Not Handled?
 - a. Checkpoint

F. **Review Questions**

1. Multiple Choice
2. True or False
3. Short Answer
4. Algorithm Workbench

G. Programming Exercises

3. Chapter 7 Lists and Tuples

A. Topics

B. 7.1 Sequences

C. 7.2 Introduction to Lists

1. The Repetition Operator
2. Iterating over a List with the for Loop
3. Indexing
4. The len Function
5. Lists Are Mutable
6. Concatenating Lists
 - a. Checkpoint

D. 7.3 List Slicing

1. Checkpoint

E. 7.4 Finding Items in Lists with the in Operator

1. Checkpoint

F. 7.5 List Methods and Useful Built-in Functions

1. The `del` Statement
2. The `min` and `max` Functions
 - a. Checkpoint

G. 7.6 Copying Lists

H. 7.7 Processing Lists

1. Totaling the Values in a List

2. **Averaging the Values in a List**
3. **Passing a List as an Argument to a Function**
4. **Returning a List from a Function**
5. **Working with Lists and Files**

I. **7.8 Two-Dimensional Lists**

1. **Checkpoint**

J. **7.9 Tuples**

1. **What's the Point?**
2. **Converting Between Lists and Tuples**
 - a. **Checkpoint**

K. **7.10 Plotting List Data with the `matplotlib` Package**

1. **Importing the `pyplot` Module**
2. **Plotting a Line Graph**
 - a. **Adding a Title, Axis Labels, and a Grid**
 - b. **Customizing the X and Y Axes**
 - c. **Displaying Markers at the Data Points**
3. **Plotting a Bar Chart**
 - a. **Customizing the Bar Width**
 - b. **Changing the Colors of the Bars**
Adding a Title, Axis Labels, and Customizing the
 - c. **Tick Mark Labels**
4. **Plotting a Pie Chart**
 - a. **Displaying Slice Labels and a Chart Title**

- b. **Changing the Colors of the Slices**
 - a. **Checkpoint**

L. **Review Questions**

- 1. **Multiple Choice**
- 2. **True or False**
- 3. **Short Answer**
- 4. **Algorithm Workbench**

M. **Programming Exercises**

4. **Chapter 8 More About Strings**

A. **Topics**

B. **8.1 Basic String Operations**

- 1. **Accessing the Individual Characters in a String**
 - a. **Iterating over a String with the for Loop**
 - b. **Indexing**
 - c. `IndexError` **Exceptions**
 - d. **The `len` Function**

- 2. **String Concatenation**
- 3. **Strings Are Immutable**
 - a. **Checkpoint**

C. **8.2 String Slicing**

- 1. **Checkpoint**

D. **8.3 Testing, Searching, and Manipulating Strings**

1. **Testing Strings with `in` and `not in`**

2. **String Methods**

- a. **String Testing Methods**
- b. **Modification Methods**
- c. **Searching and Replacing**

3. **The Repetition Operator**

4. **Splitting a String**

- a. **Checkpoint**

E. **Review Questions**

- 1. **Multiple Choice**
- 2. **True or False**
- 3. **Short Answer**
- 4. **Algorithm Workbench**

F. **Programming Exercises**

5. **Chapter 9 Dictionaries and Sets**

A. **Topics**

B. **9.1 Dictionaries**

- 1. **Creating a Dictionary**
- 2. **Retrieving a Value from a Dictionary**
Using the `in` and `not in` Operators to Test for a Value in
- 3. **a Dictionary**
- 4. **Adding Elements to an Existing Dictionary**
- 5. **Deleting Elements**
- 6. **Getting the Number of Elements in a Dictionary**

7. **Mixing Data Types in a Dictionary**
8. **Creating an Empty Dictionary**
9. **Using the `for` Loop to Iterate over a Dictionary**
10. **Some Dictionary Methods**
 - a. **The `clear` Method**
 - b. **The `get` Method**
 - c. **The `items` Method**
 - d. **The `keys` Method**
 - e. **The `pop` Method**
 - f. **The `popitem` Method**
 - g. **The `values` Method**
 - a. **Checkpoint**

C. **9.2 Sets**

1. **Creating a Set**
2. **Getting the Number of Elements in a Set**
3. **Adding and Removing Elements**
4. **Using the `for` Loop to Iterate over a Set**
 - Using the `in` and `not in` Operators to Test for a Value in**
5. **a Set**
6. **Finding the Union of Sets**
7. **Finding the Intersection of Sets**
8. **Finding the Difference of Sets**
9. **Finding the Symmetric Difference of Sets**
10. **Finding Subsets and Supersets**
 - a. **Checkpoint**

D. **9.3 Serializing Objects**

1. **Checkpoint**

E. **Review Questions**

1. **Multiple Choice**
2. **True or False**
3. **Short Answer**
4. **Algorithm Workbench**

F. **Programming Exercises**

6. **Chapter 10 Classes and Object-Oriented Programming**

A. **Topics**

B. **10.1 Procedural and Object-Oriented Programming**

1. **Object Reusability**
2. **An Everyday Example of an Object**
 - a. **Checkpoint**

C. **10.2 Classes**

1. **Class Definitions**
2. **Hiding Attributes**
3. **Storing Classes in Modules**
4. **The `BankAccount` Class**
5. **The `__str__` Method**
 - a. **Checkpoint**

D. **10.3 Working with Instances**

1. **Accessor and Mutator Methods**

2. Passing Objects as Arguments

a. Checkpoint

E. 10.4 Techniques for Designing Classes

1. The Unified Modeling Language

2. Finding the Classes in a Problem

a. Writing a Description of the Problem Domain

b. Identify All of the Nouns

c. Refining the List of Nouns

3. Identifying a Class's Responsibilities

a. The `Customer` Class

b. The `Car` Class

c. The `ServiceQuote` Class

4. This Is only the Beginning

a. Checkpoint

F. Review Questions

1. Multiple Choice

2. True or False

3. Short Answer

4. Algorithm Workbench

G. Programming Exercises

7. Chapter 11 Inheritance

A. Topics

- B. **11.1 Introduction to Inheritance**
 - 1. **Generalization and Specialization**
 - 2. **Inheritance and the “Is a” Relationship**
 - 3. **Inheritance in UML Diagrams**
 - a. **Checkpoint**
 - C. **11.2 Polymorphism**
 - 1. **The `isinstance` Function**
 - a. **Checkpoint**
 - D. **Review Questions**
 - 1. **Multiple Choice**
 - 2. **True or False**
 - 3. **Short Answer**
 - 4. **Algorithm Workbench**
 - E. **Programming Exercises**
8. **Chapter 12 Recursion**
- A. **Topics**
 - B. **12.1 Introduction to Recursion**
 - C. **12.2 Problem Solving with Recursion**
 - 1. **Using Recursion to Calculate the Factorial of a Number**
 - 2. **Direct and Indirect Recursion**
 - a. **Checkpoint**
 - D. **12.3 Examples of Recursive Algorithms**
 - 1. **Summing a Range of List Elements with Recursion**

2. **The Fibonacci Series**
3. **Finding the Greatest Common Divisor**
4. **The Towers of Hanoi**
5. **Recursion versus Looping**

E. Review Questions

1. **Multiple Choice**
2. **True or False**
3. **Short Answer**
4. **Algorithm Workbench**

F. Programming Exercises

9. Chapter 13 GUI Programming

A. Topics

B. 13.1 Graphical User Interfaces

1. **GUI Programs Are Event-Driven**
 - a. **Checkpoint**

C. 13.2 Using the `tkinter` Module

1. **Checkpoint**

D. 13.3 Display Text with `Label` Widgets

1. **Checkpoint**

E. 13.4 Organizing Widgets with `Frames`

F. 13.5 `Button` Widgets and Info Dialog Boxes

1. **Creating a Quit Button**

- G. **13.6 Getting Input with the Entry Widget**
- H. **13.7 Using Labels as Output Fields**
 - 1. **Checkpoint**
- I. **13.8 Radio Buttons and Check Buttons**
 - 1. **Radio Buttons**
 - 2. **Using Callback Functions with Radiobuttons**
 - 3. **Check Buttons**
 - a. **Checkpoint**
- J. **13.9 Drawing Shapes with the `Canvas` Widget**
 - 1. **The `Canvas` Widget's Screen Coordinate System**
 - 2. **Drawing Lines: The `create_line` Method**
 - 3. **Drawing Rectangles: The `create_rectangle` Method**
 - 4. **Drawing Ovals: The `create_oval` Method**
 - 5. **Drawing Arcs: The `create_arc` Method**
 - 6. **Drawing Polygons: The `create_polygon` Method**
 - 7. **Drawing Text: The `create_text` Method**
 - a. **Setting the Font**
 - a. **Checkpoint**
- K. **Review Questions**
 - 1. **Multiple Choice**
 - 2. **True or False**
 - 3. **Short Answer**
 - 4. **Algorithm Workbench**

L. Programming Exercises

10. Appendix A Installing Python

- A. Downloading Python**
- B. Installing Python 3.x For Windows**

11. Appendix B Introduction to IDLE

- A. Starting IDLE and Using the Python Shell**
- B. Writing a Python Program in the IDLE Editor**
- C. Color Coding**
- D. Automatic Indentation**
- E. Saving a Program**
- F. Running a Program**
- G. Other Resources**

12. Appendix C The ASCII Character Set

13. Appendix D Predefined Named Colors

14. Appendix E More About the `import` Statement

- A. Importing a Specific Function or Class**
- B. Wildcard Imports**
- C. Using an Alias**

15. Appendix F Installing Modules with the `pip` Utility

16. Appendix G Answers to Checkpoints

17. Index

- A. Symbols**
- B. A**
- C. B**

D. **C**
E. **D**
F. **E**
G. **F**
H. **G**
I. **H**
J. **I**
K. **J**
L. **K**
M. **L**
N. **M**
O. **N**
P. **O**
Q. **P**
R. **Q**
S. **R**
T. **S**
U. **T**
V. **U**
W. **V**
X. **W**
Y. **Z**

8. Credits

List of Illustrations

1. **Figure P-1 Chapter dependencies**
2. **Figure 1-1 A word processing program and an image editing program**
3. **Figure 1-2 Typical components of a computer system**
4. **Figure 1-3 The ENIAC computer**
5. **Figure 1-4 A lab technician holds a modern microprocessor**
6. **Figure 1-5 Memory chips**
7. **Figure 1-6 Think of a byte as eight switches**
8. **Figure 1-7 Bit patterns for the number 77 and the letter A**
9. **Figure 1-8 The values of binary digits as powers of 2**
10. **Figure 1-9 The values of binary digits**
11. **Figure 1-10 Determining the value of 10011101**
12. **Figure 1-11 The bit pattern for 157**
13. **Figure 1-12 Two bytes used for a large number**
14. **Figure 1-13 The letter A is stored in memory as the number 65**
15. **Figure 1-14 A digital image is stored in binary format**
16. **Figure 1-15 A program is copied into main memory and then executed**
17. **Figure 1-16 The fetch-decode-execute cycle**
18. **Figure 1-17 An assembler translates an assembly language program to a machine language program**
19. **Figure 1-18 Compiling a high-level program and executing it**
20. **Figure 1-19 Executing a high-level program with an interpreter**
21. **Figure 1-20 IDLE**

22. **Figure 2-1 The program development cycle**
23. **Figure 2-2 Flowchart for the pay calculating program**
24. **Figure 2-3 The input, processing, and output of the pay calculating program**
25. **Figure 2-4 The age variable references the value 25**
26. **Figure 2-5 Two variables**
27. **Figure 2-6 Variable reassignment in Program 2-10**
28. **Figure 2-7 The variable x references an integer**
29. **Figure 2-8 The variable x references a string**
30. **Figure 2-9 Operator precedence**
31. **Figure 2-10 The turtle's graphics window**
32. **Figure 2-11 The turtle moving forward 200 pixels**
33. **Figure 2-12 The turtle's heading**
34. **Figure 2-13 The turtle turns right**
35. **Figure 2-14 The turtle turns left**
36. **Figure 2-15 Turning the turtle by 45 degree angles**
37. **Figure 2-16 Setting the turtle's heading**
38. **Figure 2-17 Raising and lowering the pen**
39. **Figure 2-18 A circle**
40. **Figure 2-19 Drawing dots**
41. **Figure 2-20 Cartesian coordinate system**
42. **Figure 2-21 Moving the turtle**
43. **Figure 2-22 Text displayed in the graphics window**
44. **Figure 2-23 Text displayed at specific locations in the graphics window**
45. **Figure 2-24 A filled circle**
46. **Figure 2-25 A filled square**
47. **Figure 2-26 Shape filled**

48. **Figure 2-27 Stars in the Orion constellation**
49. **Figure 2-28 Names of the stars**
50. **Figure 2-29 Constellation lines**
51. **Figure 2-30 Hand sketch of the Orion constellation**
52. **Figure 2-31 Orion sketch with the names of the stars**
53. **Figure 2-32 Orion sketch with the names of the stars and constellation lines**
54. **Figure 2-33 Output of the orion.py program**
55. **Figure 2-34 Designs**
56. **Figure 3-1 A simple decision structure**
57. **Figure 3-2 A decision structure that performs three actions if it is cold outside**
58. **Figure 3-3 Example decision structure**
59. **Figure 3-4 Example decision structure**
60. **Figure 3-5 A dual alternative decision structure**
61. **Figure 3-6 Conditional execution in an if-else statement**
62. **Figure 3-7 Indentation with an if-else statement**
63. **Figure 3-8 Character codes for the strings 'Mary' and 'Mark'**
64. **Figure 3-9 Comparing each character in a string**
65. **Figure 3-10 Combining sequence structures with a decision structure**
66. **Figure 3-11 A sequence structure nested inside a decision structure**
67. **Figure 3-12 A nested decision structure**
68. **Figure 3-13 Alignment of if and else clauses**
69. **Figure 3-14 Nested blocks**
70. **Figure 3-15 Nested decision structure to determine a grade**
71. **Figure 3-16 Hit the target game**

- 72. **Figure 3-17 Missing the target**
- 73. **Figure 3-18 Hitting the target**
- 74. **Figure 3-19 Troubleshooting a bad Wi-Fi connection**
- 75. **Figure 4-1 The logic of a while loop**
- 76. **Figure 4-2 The while loop**
- 77. **Figure 4-3 Flowchart for Program 4-1**
- 78. **Figure 4-4 The for loop**
- 79. **Figure 4-5 Flowchart for Program 4-8**
- 80. **Figure 4-6 Logic for calculating a running total**
- 81. **Figure 4-7 Logic containing an input validation loop**
- 82. **Figure 4-8 Flowchart for a clock simulator**
- 83. **Figure 4-9 Octagon**
- 84. **Figure 4-10 Concentric circles**
- 85. **Figure 4-11 A design created with circles**
- 86. **Figure 4-12 Design created by Program 4-23**
- 87. **Figure 4-13 Repeating squares**
- 88. **Figure 4-14 Star pattern**
- 89. **Figure 4-15 Hypnotic pattern**
- 90. **Figure 5-1 Using functions to divide and conquer a large task**
- 91. **Figure 5-2 The function definition and the function call**
- 92. **Figure 5-3 Calling the main function**
- 93. **Figure 5-4 Calling the message function**
- 94. **Figure 5-5 The message function returns**
- 95. **Figure 5-6 The main function returns**
- 96. **Figure 5-7 All of the statements in a block are indented**
- 97. **Figure 5-8 Function call symbol**
- 98. **Figure 5-9 Flowchart for Program 5-2**

- 99. **Figure 5-10 A hierarchy chart**
- 100. **Figure 5-11 Hierarchy chart for the program**
- 101. **Figure 5-12 Each function has its own birds variable**
- 102. **Figure 5-13 The value variable is passed as an argument**
- 103. **Figure 5-14 The value variable and the number parameter reference the same value**
- 104. **Figure 5-15 Hierarchy chart for the program**
- 105. **Figure 5-16 Two arguments passed to two parameters**
- 106. **Figure 5-17 The value variable is passed to the change_me function**
- 107. **Figure 5-18 The value variable is passed to the change_me function**
- 108. **Figure 5-19 A library function viewed as a black box**
- 109. **Figure 5-20 A statement that calls the random function**
- 110. **Figure 5-21 The random function returns a value**
- 111. **Figure 5-22 Displaying a random number**
- 112. **Figure 5-23 Parts of the function**
- 113. **Figure 5-24 Arguments are passed to the sum function and a value is returned**
- 114. **Figure 5-25 IPO charts for the getRegularPrice and discount functions**
- 115. **Figure 5-26 Output of Program 5-29**
- 116. **Figure 5-27 Output of Program 5-30**
- 117. **Figure 5-28 Output of Program 5-31**
- 118. **Figure 5-29 Output of Program 5-33**
- 119. **Figure 5-30 Snowman**
- 120. **Figure 5-31 Rectangular pattern**
- 121. **Figure 5-32 Checkerboard pattern**

- 122. **Figure 5-33 City skyline**
- 123. **Figure 6-1 Writing data to a file**
- 124. **Figure 6-2 Reading data from a file**
- 125. **Figure 6-3 Three files**
- 126. **Figure 6-4 A variable name references a file object that is associated with a file**
- 127. **Figure 6-5 Contents of the file philosophers.txt**
- 128. **Figure 6-6 Contents of philosophers.txt in Notepad**
- 129. **Figure 6-7 The file_contents variable references the string that was read from the file**
- 130. **Figure 6-8 Initial read position**
- 131. **Figure 6-9 Read position advanced to the next line**
- 132. **Figure 6-10 Read position advanced to the next line**
- 133. **Figure 6-11 Read position advanced to the end of the file**
- 134. **Figure 6-12 The strings referenced by the line1, line2, and line3 variables**
- 135. **Figure 6-13 The friends.txt file**
- 136. **Figure 6-14 Contents of the numbers.txt file**
- 137. **Figure 6-15 The numbers.txt file viewed in Notepad**
- 138. **Figure 6-16 Contents of the sales.txt file**
- 139. **Figure 6-17 General logic for detecting the end of a file**
- 140. **Figure 6-18 Fields in a record**
- 141. **Figure 6-19 Records in a file**
- 142. **Figure 6-20 Handling an exception**
- 143. **Figure 7-1 A list of integers**
- 144. **Figure 7-2 A list of strings**
- 145. **Figure 7-3 A list holding different types**
- 146. **Figure 7-4 list1 and list2 reference the same list**

- 147. **Figure 7-5 A two-dimensional list**
- 148. **Figure 7-6 Two-dimensional list with three rows and three columns**
- 149. **Figure 7-7 Subscripts for each element of the scores list**
- 150. **Figure 7-8 Output of Program 7-19**
- 151. **Figure 7-9 Output of Program 7-20**
- 152. **Figure 7-10 Output of Program 7-21**
- 153. **Figure 7-11 Output of Program 7-22**
- 154. **Figure 7-12 Output of Program 7-23**
- 155. **Figure 7-13 Output of Program 7-24**
- 156. **Figure 7-14 Output of Program 7-25**
- 157. **Figure 7-15 Output of Program 7-26**
- 158. **Figure 7-16 Output of Program 7-27**
- 159. **Figure 7-17 Output of Program 7-28**
- 160. **Figure 7-18 The Lo Shu Magic Square**
- 161. **Figure 7-19 The sum of the rows, columns, and diagonals**
- 162. **Figure 8-1 Iterating over the string 'Juliet'**
- 163. **Figure 8-2 String indexes**
- 164. **Figure 8-3 Getting a copy of a character from a string**
- 165. **Figure 8-4 The string 'Carmen' assigned to name**
- 166. **Figure 8-5 The string 'Carmen Brown' assigned to name**
- 167. **Figure 8-6 The pbnumbers.txt file**
- 168. **Figure 8-7 The GasPrices.txt file**
- 169. **Figure 9-1 Example of original file and index file**
- 170. **Figure 10-1 An object contains data attributes and methods**
- 171. **Figure 10-2 Code outside the object interacts with the object's methods**
- 172. **Figure 10-3 A blueprint and houses built from the blueprint**

173. **Figure 10-4 The cookie cutter metaphor**
174. **Figure 10-5 The housefly and mosquito objects are instances of the Insect class**
175. **Figure 10-6 Actions caused by the Coin() expression**
176. **Figure 10-7 The my_coin variable references a Coin object**
177. **Figure 10-8 The coin1, coin2, and coin3 variables reference three Coin objects**
178. **Figure 10-9 The objects after the toss method**
179. **Figure 10-10 General layout of a UML diagram for a class**
180. **Figure 10-11 UML diagram for the Coin class**
181. **Figure 10-12 UML diagram for the CellPhone class**
182. **Figure 10-13 UML diagram for the Customer class**
183. **Figure 10-14 UML diagram for the Car class**
184. **Figure 10-15 UML diagram for the ServiceQuote class**
185. **Figure 11-1 Bumblebees and grasshoppers are specialized versions of an insect**
186. **Figure 11-2 UML diagram showing inheritance**
187. **Figure 12-1 First two calls of the function**
188. **Figure 12-2 Six calls to the message function**
189. **Figure 12-3 Control returns to the point after the recursive function call**
190. **Figure 12-4 The value of num and the return value during each call of the function**
191. **Figure 12-5 The pegs and discs in the Tower of Hanoi game**
192. **Figure 12-6 Steps for moving three pegs**
193. **Figure 13-1 A command line interface**
194. **Figure 13-2 A dialog box**
195. **Figure 13-3 A GUI program**

- 196. **Figure 13-4 Window displayed by Program 13-1**
- 197. **Figure 13-5 Window displayed by Program 13-3**
- 198. **Figure 13-6 Window displayed by Program 13-4**
- 199. **Figure 13-7 Window displayed by Program 13-5**
- 200. **Figure 13-8 Window displayed by Program 13-6**
- 201. **Figure 13-9 Arrangement of widgets**
- 202. **Figure 13-10 The main window displayed by Program 13-7**
- 203. **Figure 13-11 The info dialog box displayed by Program 13-7**
- 204. **Figure 13-12 The info dialog box displayed by Program 13-8**
- 205. **Figure 13-13 The kilo_converter program's window**
- 206. **Figure 13-14 The window organized with frames**
- 207. **Figure 13-15 The info dialog box**
- 208. **Figure 13-16 The window initially displayed**
- 209. **Figure 13-17 The window showing 1000 kilometers converted to miles**
- 210. **Figure 13-18 Layout of the kilo_converter2 program's main window**
- 211. **Figure 13-19 A sketch of the window**
- 212. **Figure 13-20 Using Frames to organize the widgets**
- 213. **Figure 13-21 The test_averages program window**
- 214. **Figure 13-22 A group of radio buttons**
- 215. **Figure 13-23 Window displayed by Program 13-12**
- 216. **Figure 13-24 A group of check buttons**
- 217. **Figure 13-25 Window displayed by Program 13-13**
- 218. **Figure 13-26 Various pixel locations in a 640 by 480 window**

- 219. **Figure 13-27 Window displayed by Program 13-14**
- 220. **Figure 13-28 Window displayed by Program 13-15**
- 221. **Figure 13-29 Window displayed by Program 13-16**
- 222. **Figure 13-30 Dashed, 3-pixel wide outline**
- 223. **Figure 13-31 An oval's bounding rectangle**
- 224. **Figure 13-32 Window displayed by Program 13-17**
- 225. **Figure 13-33 Arc properties**
- 226. **Figure 13-34 Window displayed by Program 13-18**
- 227. **Figure 13-35 Types of arcs**
- 228. **Figure 13-36 Window displayed by Program 13-19**
- 229. **Figure 13-37 Points of each vertex in the polygon**
- 230. **Figure 13-38 Window displayed by Program 13-20**
- 231. **Figure 13-39 Window displayed by Program 13-21**
- 232. **Figure 13-40 Results of the various anchor values**
- 233. **Figure 13-41 Window displayed by Program 13-22**
- 234. **Figure 13-42 Name and address program**
- 235. **Figure 13-43 Hollywood star**
- 236. **Figure A-1 Download the latest Python version**
- 237. **Figure A-2 Python installer**
- 238. **Figure B-1 IDLE shell window**
- 239. **Figure B-2 Statements executed by the Python interpreter**
- 240. **Figure B-3 A multiline statement executed by the Python interpreter**
- 241. **Figure B-4 The File menu**
- 242. **Figure B-5 A text editing window**
- 243. **Figure B-6 Lines that cause automatic indentation**
- 244. **Figure B-7 The editor window's Run menu**
- 245. **Figure B-8 Save confirmation dialog box**

- 246. **Figure B-9 Output displayed in the Python Shell window**
- 247. **Figure B-10 Dialog box reporting a syntax error**

List of Tables

- 1. **Table 1-1 Programming languages**
- 2. **Table 1-2 The Python key words**
- 3. **Table 2-1 Sample variable names**
- 4. **Table 2-2 Data conversion functions**
- 5. **Table 2-3 Python math operators**
- 6. **Table 2-4 Some expressions**
- 7. **Table 2-5 More expressions and their values**
- 8. **Table 2-6 Algebraic expressions**
- 9. **Table 2-7 Algebraic and programming expressions**
- 10. **Table 2-8 Some of Python's escape characters**
- 11. **Table 3-1 Relational operators**
- 12. **Table 3-2 Boolean expressions using relational operators**
- 13. **Table 3-3 Logical operators**
- 14. **Table 3-4 Compound Boolean expressions using logical operators**
- 15. **Table 3-5 Truth table for the and operator**
- 16. **Table 3-6 Truth table for the or operator**
- 17. **Table 3-7 Truth table for the not operator**
- 18. **Table 4-1 Various assignment statements (assume x = 6 in each statement)**
- 19. **Table 4-2 Augmented assignment operators**
- 20. **Table 5-1 Sales commission rates**

21. **Table 5-2 Many of the functions in the math module**
22. **Table 6-1 Some of the Python file modes**
23. **Table 7-1 A few of the list methods**
24. **Table 7-2 Some of the marker symbols**
25. **Table 7-3 Color codes**
26. **Table 8-1 Some string testing methods**
27. **Table 8-2 String Modification Methods**
28. **Table 8-3 Search and replace methods**
29. **Table 8-4 Morse code**
30. **Table 9-1 Some of the dictionary methods**
31. **Table 13-1 tkinter widgets**
32. **Table 13-2 Some of the optional arguments to the
create_line method**
33. **Table 13-3 Some of the optional arguments to the
create_rectangle method**
34. **Table 13-4 Some of the optional arguments to the
create_oval method**
35. **Table 13-5 Some of the optional arguments to the
create_arc method**
36. **Table 13-6 Arc types**
37. **Table 13-7 Some of the optional arguments to the
create_polygon method**
38. **Table 13-8 Some of the optional arguments to the
create_text method**
39. **Table 13-9 anchor values**
40. **Table 13-10 Keyword arguments for the Font class**

Landmarks

1. **Contents in a Glance**
2. **Frontmatter**
3. **Start of Content**
4. **backmatter**
5. **List of Illustrations**
6. **List of Tables**

1. **i**
2. **ii**
3. **iii**
4. **iv**
5. **v**
6. **vi**
7. **vii**
8. **viii**
9. **ix**
10. **x**
11. **xi**
12. **xii**
13. **xiii**
14. **xiv**
15. **xv**
16. **xvi**
17. **xvii**
18. **xviii**
19. **xix**

- 20. **xx**
- 21. **xxi**
- 22. **xxii**
- 23. **1**
- 24. **2**
- 25. **3**
- 26. **4**
- 27. **5**
- 28. **6**
- 29. **7**
- 30. **8**
- 31. **9**
- 32. **10**
- 33. **11**
- 34. **12**
- 35. **13**
- 36. **14**
- 37. **15**
- 38. **16**
- 39. **17**
- 40. **18**
- 41. **19**
- 42. **20**
- 43. **21**
- 44. **22**
- 45. **23**
- 46. **24**
- 47. **25**

- 48. **26**
- 49. **27**
- 50. **28**
- 51. **29**
- 52. **30**
- 53. **31**
- 54. **32**
- 55. **33**
- 56. **34**
- 57. **35**
- 58. **36**
- 59. **37**
- 60. **38**
- 61. **39**
- 62. **40**
- 63. **41**
- 64. **42**
- 65. **43**
- 66. **44**
- 67. **45**
- 68. **46**
- 69. **47**
- 70. **48**
- 71. **49**
- 72. **50**
- 73. **51**
- 74. **52**
- 75. **53**

- 76. **54**
- 77. **55**
- 78. **56**
- 79. **57**
- 80. **58**
- 81. **59**
- 82. **60**
- 83. **61**
- 84. **62**
- 85. **63**
- 86. **64**
- 87. **65**
- 88. **66**
- 89. **67**
- 90. **68**
- 91. **69**
- 92. **70**
- 93. **71**
- 94. **72**
- 95. **73**
- 96. **74**
- 97. **75**
- 98. **76**
- 99. **77**
- 100. **78**
- 101. **79**
- 102. **80**
- 103. **81**

104.	82
105.	83
106.	84
107.	85
108.	86
109.	87
110.	88
111.	89
112.	90
113.	91
114.	92
115.	93
116.	94
117.	95
118.	96
119.	97
120.	98
121.	99
122.	100
123.	101
124.	102
125.	103
126.	104
127.	105
128.	106
129.	107
130.	108
131.	109

132. **110**
133. **111**
134. **112**
135. **113**
136. **114**
137. **115**
138. **116**
139. **117**
140. **118**
141. **119**
142. **120**
143. **121**
144. **122**
145. **123**
146. **124**
147. **125**
148. **126**
149. **127**
150. **128**
151. **129**
152. **130**
153. **131**
154. **132**
155. **133**
156. **134**
157. **135**
158. **136**
159. **137**

160.	138
161.	139
162.	140
163.	141
164.	142
165.	143
166.	144
167.	145
168.	146
169.	147
170.	148
171.	149
172.	150
173.	151
174.	152
175.	153
176.	154
177.	155
178.	156
179.	157
180.	158
181.	159
182.	160
183.	161
184.	162
185.	163
186.	164
187.	165

188.	166
189.	167
190.	168
191.	169
192.	170
193.	171
194.	172
195.	173
196.	174
197.	175
198.	176
199.	177
200.	178
201.	179
202.	180
203.	181
204.	182
205.	183
206.	184
207.	185
208.	186
209.	187
210.	188
211.	189
212.	190
213.	191
214.	192
215.	193

216.	194
217.	195
218.	196
219.	197
220.	198
221.	199
222.	200
223.	201
224.	202
225.	203
226.	204
227.	205
228.	206
229.	207
230.	208
231.	209
232.	210
233.	211
234.	212
235.	213
236.	214
237.	215
238.	216
239.	217
240.	218
241.	219
242.	220
243.	221

244. **222**
245. **223**
246. **224**
247. **225**
248. **226**
249. **227**
250. **228**
251. **229**
252. **230**
253. **231**
254. **232**
255. **233**
256. **234**
257. **235**
258. **236**
259. **237**
260. **238**
261. **239**
262. **240**
263. **241**
264. **242**
265. **243**
266. **244**
267. **245**
268. **246**
269. **247**
270. **248**
271. **249**

272. **250**
273. **251**
274. **252**
275. **253**
276. **254**
277. **255**
278. **256**
279. **257**
280. **258**
281. **259**
282. **260**
283. **261**
284. **262**
285. **263**
286. **264**
287. **265**
288. **266**
289. **267**
290. **268**
291. **269**
292. **270**
293. **271**
294. **272**
295. **273**
296. **274**
297. **275**
298. **276**
299. **277**

300. **278**
301. **279**
302. **280**
303. **281**
304. **282**
305. **283**
306. **284**
307. **285**
308. **286**
309. **287**
310. **288**
311. **289**
312. **290**
313. **291**
314. **292**
315. **293**
316. **294**
317. **295**
318. **296**
319. **297**
320. **298**
321. **299**
322. **300**
323. **301**
324. **302**
325. **303**
326. **304**
327. **305**

328. **306**
329. **307**
330. **308**
331. **309**
332. **310**
333. **311**
334. **312**
335. **313**
336. **314**
337. **315**
338. **316**
339. **317**
340. **318**
341. **319**
342. **320**
343. **321**
344. **322**
345. **323**
346. **324**
347. **325**
348. **326**
349. **327**
350. **328**
351. **329**
352. **330**
353. **331**
354. **332**
355. **333**

356. **334**
357. **335**
358. **336**
359. **337**
360. **338**
361. **339**
362. **340**
363. **341**
364. **342**
365. **343**
366. **344**
367. **345**
368. **346**
369. **347**
370. **348**
371. **349**
372. **350**
373. **351**
374. **352**
375. **353**
376. **354**
377. **355**
378. **356**
379. **357**
380. **358**
381. **359**
382. **360**
383. **361**

- 384. **362**
- 385. **363**
- 386. **364**
- 387. **365**
- 388. **366**
- 389. **367**
- 390. **368**
- 391. **369**
- 392. **370**
- 393. **371**
- 394. **372**
- 395. **373**
- 396. **374**
- 397. **375**
- 398. **376**
- 399. **377**
- 400. **378**
- 401. **379**
- 402. **380**
- 403. **381**
- 404. **382**
- 405. **383**
- 406. **384**
- 407. **385**
- 408. **386**
- 409. **387**
- 410. **388**
- 411. **389**

412. **390**
413. **391**
414. **392**
415. **393**
416. **394**
417. **395**
418. **396**
419. **397**
420. **398**
421. **399**
422. **400**
423. **401**
424. **402**
425. **403**
426. **404**
427. **405**
428. **406**
429. **407**
430. **408**
431. **409**
432. **410**
433. **411**
434. **412**
435. **413**
436. **414**
437. **415**
438. **416**
439. **417**

440. **418**
441. **419**
442. **420**
443. **421**
444. **422**
445. **423**
446. **424**
447. **425**
448. **426**
449. **427**
450. **428**
451. **429**
452. **430**
453. **431**
454. **432**
455. **433**
456. **434**
457. **435**
458. **436**
459. **437**
460. **438**
461. **439**
462. **440**
463. **441**
464. **442**
465. **443**
466. **444**
467. **445**

468. **446**
469. **447**
470. **448**
471. **449**
472. **450**
473. **451**
474. **452**
475. **453**
476. **454**
477. **455**
478. **456**
479. **457**
480. **458**
481. **459**
482. **460**
483. **461**
484. **462**
485. **463**
486. **464**
487. **465**
488. **466**
489. **467**
490. **468**
491. **469**
492. **470**
493. **471**
494. **472**
495. **473**

496. **474**
497. **475**
498. **476**
499. **477**
500. **478**
501. **479**
502. **480**
503. **481**
504. **482**
505. **483**
506. **484**
507. **485**
508. **486**
509. **487**
510. **488**
511. **489**
512. **490**
513. **491**
514. **492**
515. **493**
516. **494**
517. **495**
518. **496**
519. **497**
520. **498**
521. **499**
522. **500**
523. **501**

524. **502**
525. **503**
526. **504**
527. **505**
528. **506**
529. **507**
530. **508**
531. **509**
532. **510**
533. **511**
534. **512**
535. **513**
536. **514**
537. **515**
538. **516**
539. **517**
540. **518**
541. **519**
542. **520**
543. **521**
544. **522**
545. **523**
546. **524**
547. **525**
548. **526**
549. **527**
550. **528**
551. **529**

552. **530**
553. **531**
554. **532**
555. **533**
556. **534**
557. **535**
558. **536**
559. **537**
560. **538**
561. **539**
562. **540**
563. **541**
564. **542**
565. **543**
566. **544**
567. **545**
568. **546**
569. **547**
570. **548**
571. **549**
572. **550**
573. **551**
574. **552**
575. **553**
576. **554**
577. **555**
578. **556**
579. **557**

580. **558**
581. **559**
582. **560**
583. **561**
584. **562**
585. **563**
586. **564**
587. **565**
588. **566**
589. **567**
590. **568**
591. **569**
592. **570**
593. **571**
594. **572**
595. **573**
596. **574**
597. **575**
598. **576**
599. **577**
600. **578**
601. **579**
602. **580**
603. **581**
604. **582**
605. **583**
606. **584**
607. **585**

- 608. **586**
- 609. **587**
- 610. **588**
- 611. **589**
- 612. **590**
- 613. **591**
- 614. **592**
- 615. **593**
- 616. **594**
- 617. **595**
- 618. **596**
- 619. **597**
- 620. **598**
- 621. **599**
- 622. **600**
- 623. **601**
- 624. **602**
- 625. **603**
- 626. **604**
- 627. **605**
- 628. **606**
- 629. **607**
- 630. **608**
- 631. **609**
- 632. **610**
- 633. **611**
- 634. **612**
- 635. **613**

636. **614**
637. **615**
638. **616**
639. **617**
640. **618**
641. **619**
642. **620**
643. **621**
644. **622**
645. **623**
646. **624**
647. **625**
648. **626**
649. **627**
650. **628**
651. **629**
652. **630**
653. **631**
654. **632**
655. **633**
656. **634**
657. **635**
658. **636**
659. **637**
660. **638**
661. **639**
662. **640**
663. **641**

664. **642**
665. **643**
666. **644**
667. **645**
668. **646**
669. **647**
670. **648**
671. **649**
672. **650**
673. **651**
674. **652**
675. **653**
676. **654**
677. **655**
678. **656**
679. **657**
680. **658**
681. **659**
682. **660**
683. **661**
684. **662**
685. **663**
686. **664**
687. **665**
688. **666**
689. **667**
690. **668**
691. **669**

692. **670**
693. **671**
694. **672**
695. **673**
696. **674**
697. **675**
698. **676**
699. **677**
700. **678**
701. **679**
702. **680**
703. **681**
704. **682**
705. **683**
706. **684**
707. **685**
708. **686**
709. **687**
710. **688**
711. **689**
712. **690**
713. **691**
714. **692**
715. **693**
716. **694**
717. **695**
718. **696**
719. **697**

720. **698**
721. **699**
722. **700**
723. **701**
724. **702**
725. **703**
726. **704**
727. **705**
728. **706**
729. **707**
730. **708**
731. **709**
732. **710**
733. **711**
734. **712**
735. **713**
736. **714**
737. **715**
738. **716**
739. **717**
740. **718**
741. **719**
742. **720**
743. **721**
744. **722**